18/novembro/2026

PUCRS | ESCOLA POLITÉCNICA

4646B-04

FUNDAMENTOS DE SISTEMAS DIGITAIS

UNIDADE 4
Projeto de Final de Disciplina
NanoCPU (modelagem SV)

MARISTA

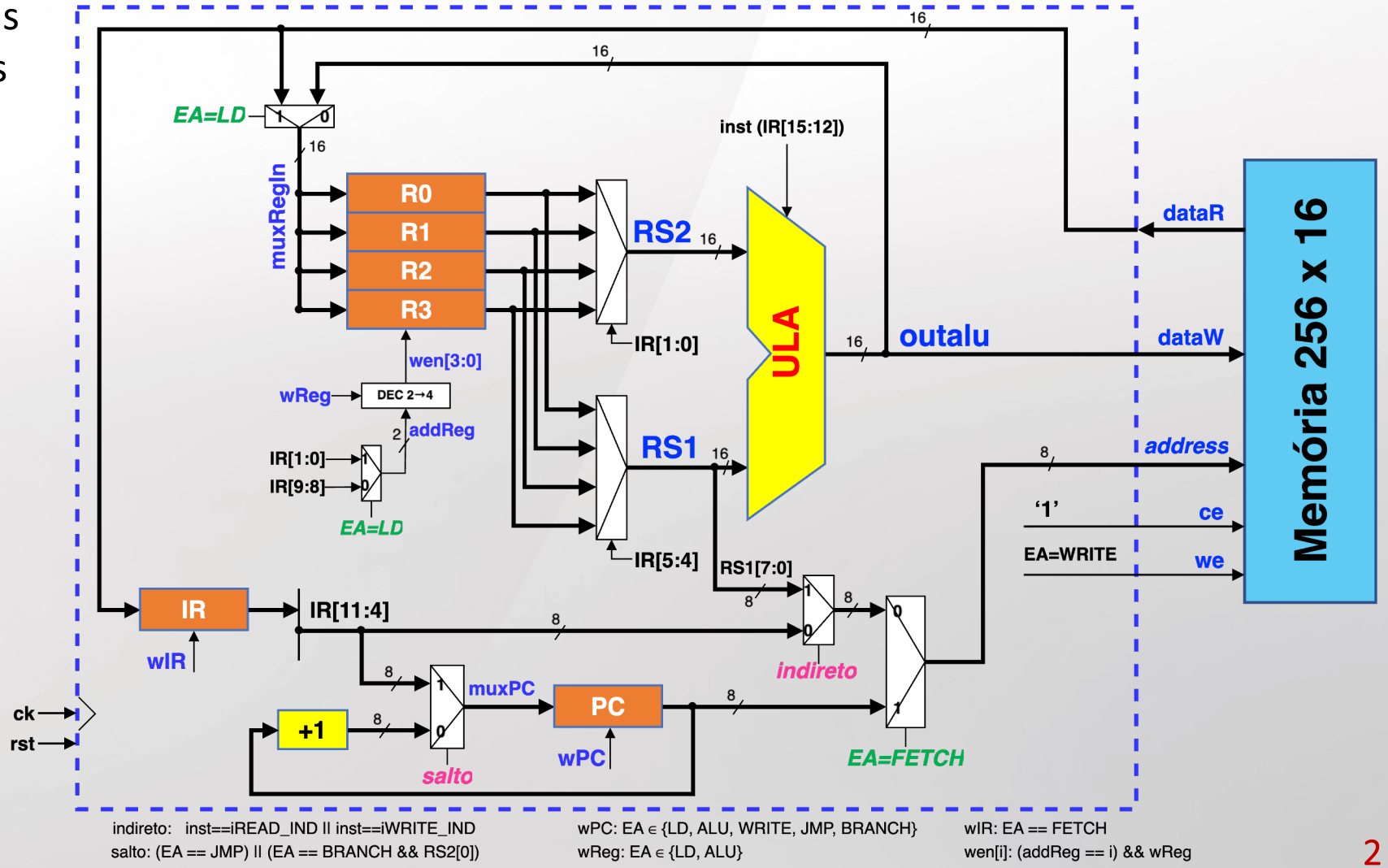
Versão com acesso indireto à memória

Nano CPU

Exemplo de processador muito simples, porém seguindo conceitos importantes em arquitetura de processadores:

1. Operações lógicas e aritméticas executadas entre registradores
   Arquitetura *load-store*
2. Instruções de formato fixo

- **R0 a R3 → registradores de propósito geral**
- **PC: *program counter* – endereço atual do programa na memória**
- **IR: *instruction register* – instrução atual**

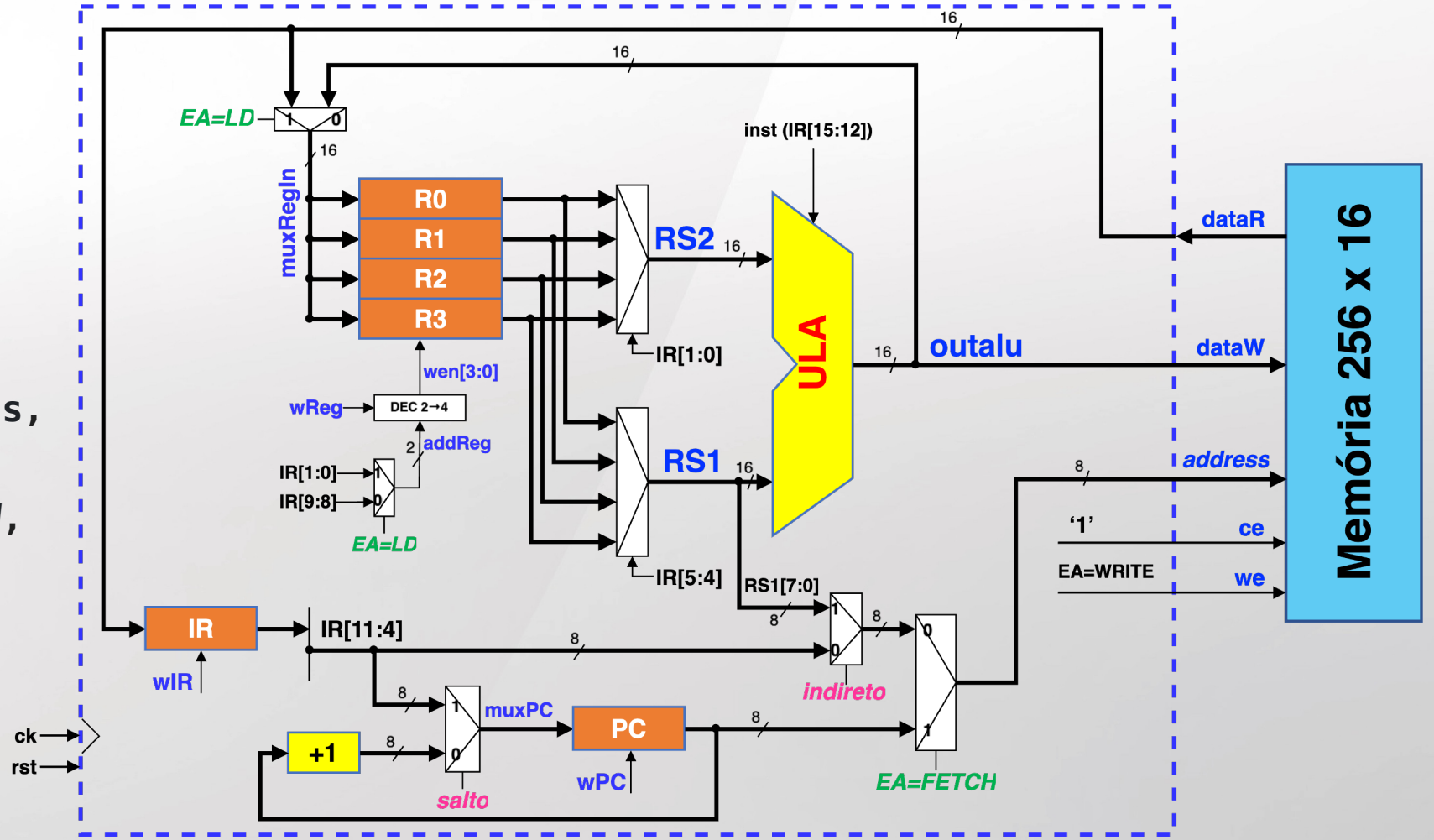


Nano CPU - Interface Externa

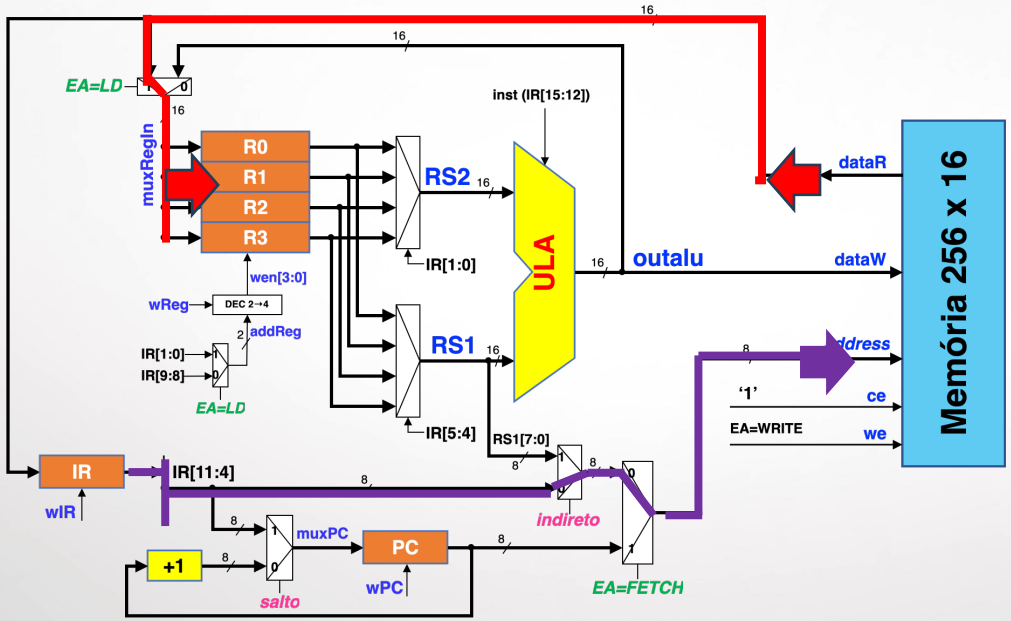
Largura da palavra: 16 bits

Capacidade da memória: 256 palavras (endereçamento de 8 bits)

```
module NanoCPU
    import nanoCPU_pkg::*;
    (
     input logic ck, rst,
     output logic [7:0] address,
     input logic [15:0] dataR,
     output logic [15:0] dataW,
     output logic ce, we
    );
```



Como processar os dados armazenados em memória?

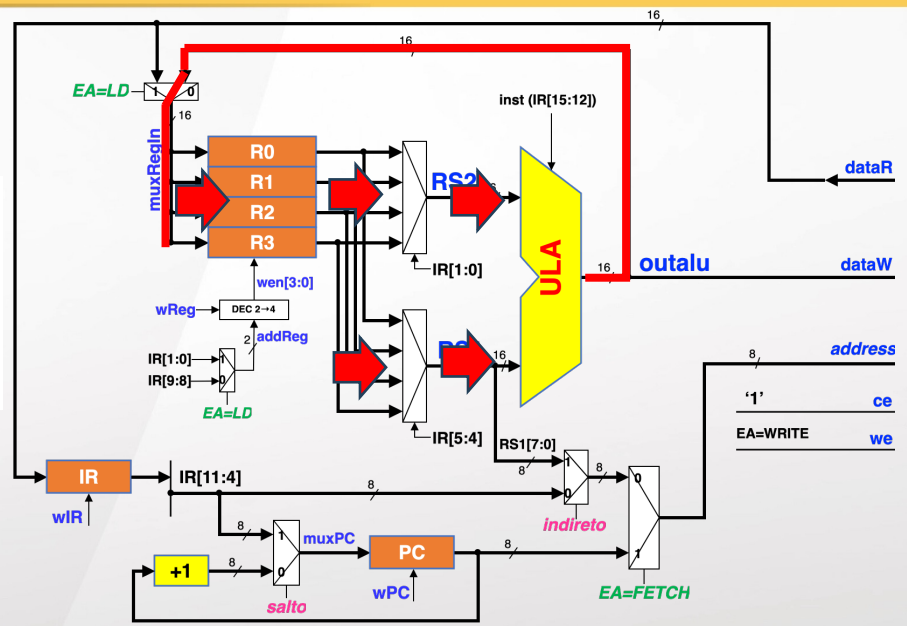


1- Le operandos da memória

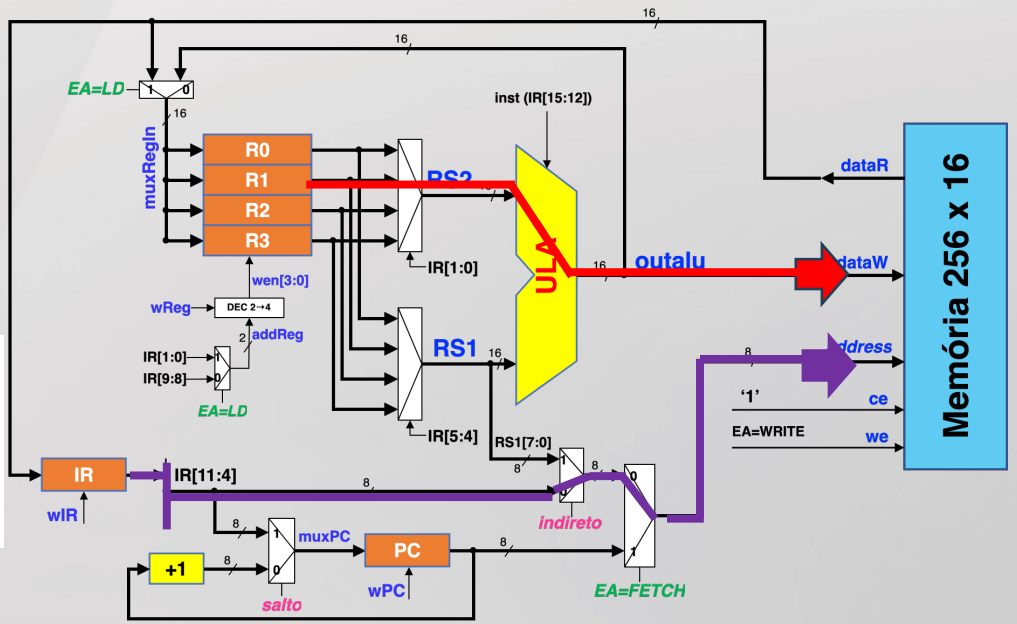
- IR contém instrução atual, e nesta instrução há o endereço da memória
- Dado lido é escrito em um registrador

2 - Operações lógico-aritméticas

R*x* ← RS1 *op* RS2



3- Escreve o conteúdo de um registrador na memória



Nano CPU – conjunto de instruções

- O registrador IR contém **instruções** de formato **fixo**:
  - Bits 15:12 – 4 bits que especificam a instrução
  - Bits 11:4 – podem indicar um endereço de 8 bits ou endereços de registradores
  - Bits 3:0 – endereço de registrador

Instrução	15:12	11:8	7:4	3:0	Operação
READ	0	endereço		Rs2	Rs2 ← PMEM(end)
WRITE	1	endereço		Rs2	PMEM(end) ← Rs2
SALTO	2	endereço		0	PC ← end
SALTO COND	3	endereço		Rs2	PC ← end **se** Rs2[0]=1
XOR	4	**Rt**	Rs1	Rs2	Rt ← Rs1 xor Rs2
SUB	5	**Rt**	Rs1	Rs2	Rt ← Rs1 - Rs2
ADD	6	**Rt**	Rs1	Rs2	Rt ← Rs1 + Rs2
MENOR	7	**Rt**	Rs1	Rs2	Rt ← 1 **se** Rs1<Rs2 senão 0
...	...				*a ser incluído pelos alunos*
READ_Indireto	$10\ (A)_{16}$	0	Rs1	Rs2	Rs2 ← PMEM(**RS1**)
WRITE_Indireto	$11\ (B)_{16}$	0	Rs1	Rs2	PMEM(**RS1**) ← Rs2
FIM	$15\ (F)_{16}$	0	0	0	termina a execução

Notar que podemos ter até 16 registradores de propósito geral – por que?

Nano CPU – (de)codificação no package nanoCPU_pkg

Instrução	15:12
READ	0
WRITE	1
SALTO	2
SALTO COND	3
XOR	4
SUB	5
ADD	6
MENOR	7
…	…
READ_Indireto	10 $(A)_{16}$
WRITE_Indireto	11 $(B)_{16}$
FIM	15 $(F)_{16}$



```
package nanoCPU_pkg;
    typedef enum logic [3:0] {
        iREAD       = 4'h0,
        iWRITE      = 4'h1,
        iJMP        = 4'h2,
        iBRANCH     = 4'h3,
        iXOR        = 4'h4,
        iSUB        = 4'h5,
        iADD        = 4'h6,
        iLESS       = 4'h7,
        iREAD_IND   = 4'hA,
        iWRITE_IND  = 4'hB,
        iEND        = 4'hF
    } instType;
endpackage
```

- **No código da nanoCPU a decodificação é feita diretamente pelo campo IR[15:12] :**

```
assign inst = instType'(IR[15:12]);  // decode the current instruction (opcode -> instruction)
```

Programando o processador

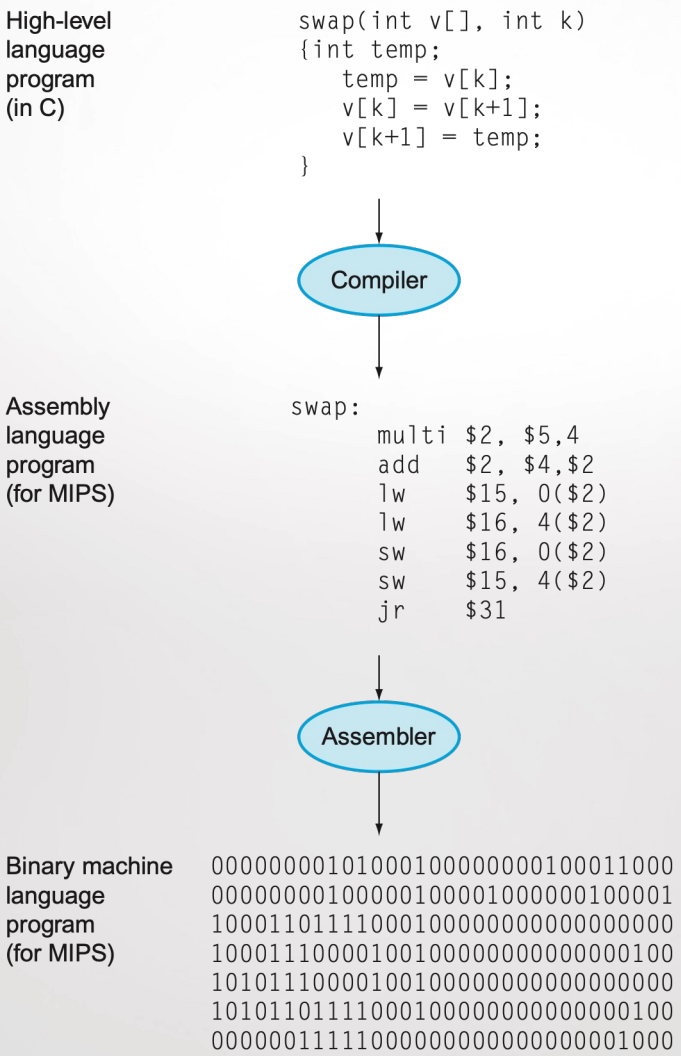


FIGURE 1.4 C program compiled into assembly language and then assembled into binary machine language. Although the translation from high-level language to binary machine language is shown in two steps, some compilers cut out the middleman and produce binary machine language directly. These languages and this program are examined in more detail in Chapter 2.

- A memória contém o programa a ser executado, assim como os dados utilizados no processamento
- Programar direto em binário é impraticável – por isto os processadores usam um linguagem chamada ***assembly***, única para cada processador

Linguagem ***assembly***: descrição de um programa utilizando instruções suportadas pela arquitetura (**ISA** – *Instruction Set Architecture)*

Código binário que vai para a memória do processador

Exemplo de programa (sintaxe informal)

```
i = 0
a =0
do
        a = a +  i
        i  = i + 1
while (i<10)
"write" a
fim
```

Endereço	Instrução - *assembly*	comentário	Binário (hexa)
0	**read** R0, 10	Le da posição 10: R0←0 (i)	0 0A 0
1	**read** R1, 10	Le da posição 10: R1←0 (a)	0 0A 1
2	**read** R3, 12	Le da posição 12: R3 ← 10	0 0C 3
3	**add** R1, R1, R0	a = a + i	6 1 1 0
4	**read** R2, 11	Le da posição 11: R2 ← 1	0 0B 2
5	**add** R0, R0, R2	i = i + 1	6 0 0 2
6	**menor** R2, R0, R3	R2←1 se i<10 else 0	7 2 0 3
7	**salto cond** 3, R2	se R2=1 salta para 3	3 0 3 2
8	**write** R1, 13	Escreve a na posição 13	1 0D 1
9	**fim**	Termina a execução	F000
10 $(A)_{16}$	0		0000
11 $(B)_{16}$	1		0001
12 $(C)_{16}$	10		000A
13 $(D)_{16}$		Receberá o valor de a	0000

Instrução	15:12	11:8	7:4	3:0	Operação
READ	0	endereço		Rs2	Rs2 ← PMEM(end)
WRITE	1	endereço		Rs2	PMEM(end) ← Rs2
SALTO	2	endereço		0	PC ← end
SALTO COND	3	endereço		Rs2	PC ← end **se** Rs2=1
XOR	4	**Rt**	Rs1	Rs2	Rt ← Rs1 xor Rs2
SUB	5	**Rt**	Rs1	Rs2	Rt ← Rs1 - Rs2
ADD	6	**Rt**	Rs1	Rs2	Rt ← Rs1 + Rs2
MENOR	7	**Rt**	Rs1	Rs2	Rt ← 1 **se** Rs1<Rs2 senão 0
...	...				*a ser incluído pelos alunos*
READ_Indireto	10 $(A)_{16}$	0	Rs1	Rs2	Rs2 ← PMEM(**RS1**)
WRITE_Indireto	11 $(B)_{16}$	0	Rs1	Rs2	PMEM(**RS1**) ← Rs2
FIM	15 $(F)_{16}$	0	0	0	termina a execução

No test bench:

```
memory_array_t memoryA = '{
    0: 'h00A0,
    1: 'h00A1,
    2: 'h00C3,
    3: 'h6110,
    4: 'h00B2,
    5: 'h6002,
    6: 'h7203,
    7: 'h3032,
    8: 'h10D1,
    9: 'hF000,
    10: 'h0000,
    11: 'h0001,
    12: 'h000A,
    13: 'h0000,
    default: 'h0000
};
```

Execução (impressão de debug):

```
Fim da simulacao:

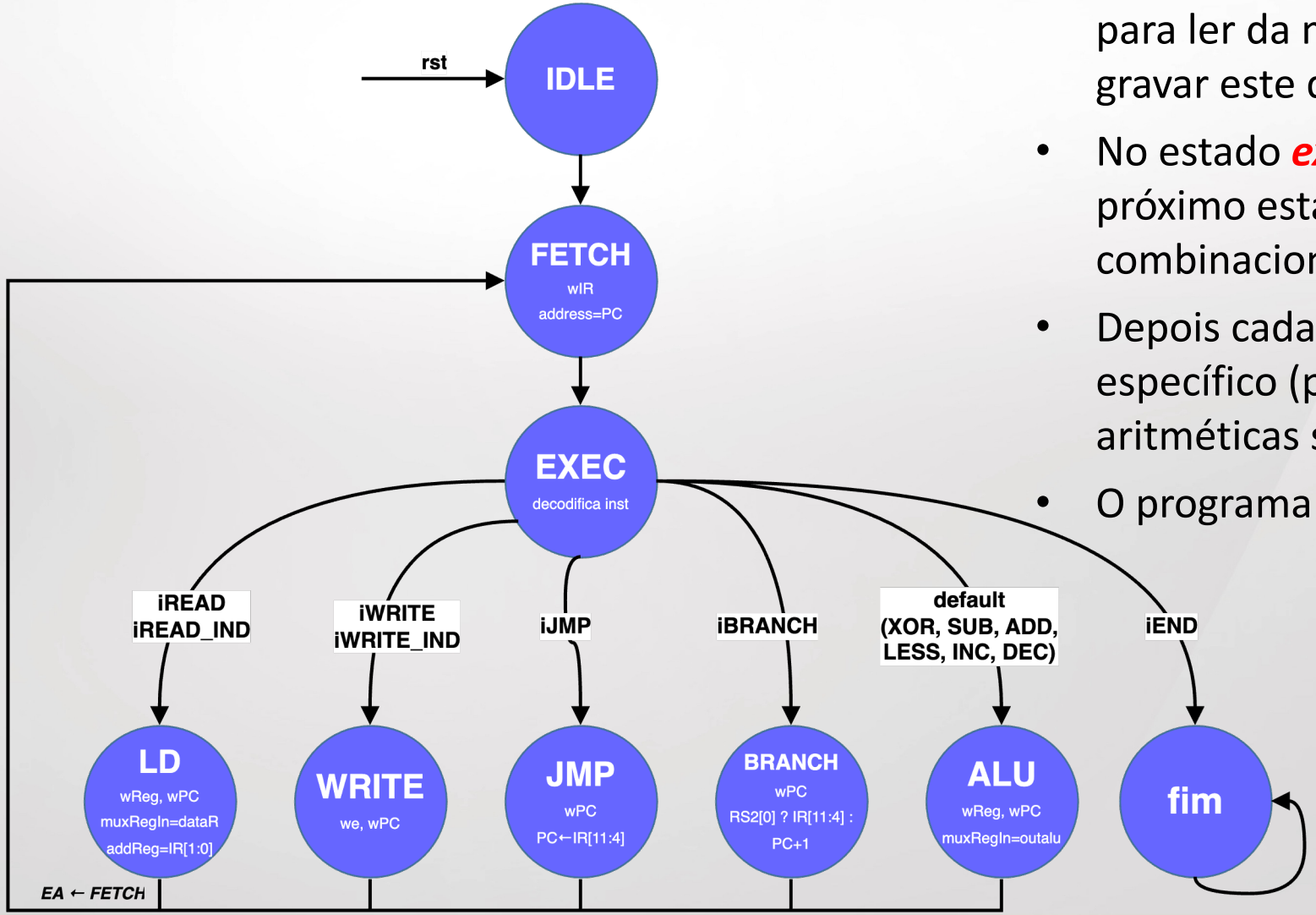
 R0 = 000a   R1 = 002d    R2 = 0000   R3 = 000a

Memoria – primeiras 60 posicoes (end: conteudo instrucao/dado):
  0: 00a0 iREAD     |   1: 00a1 iREAD     |   2: 00c3 iREAD     |   3: 6110 iADD      |   4: 00b2 iREAD
  5: 6002 iADD      |   6: 7203 iLESS     |   7: 3032 iBRANCH   |   8: 10d1 iWRITE    |   9: f000 iEND
 10: 0000 0         |  11: 0001 1         |  12: 000a 10        |  13: 002d 45        |  14: 0000 0
 …
```

- O programa simulado é escolhido no nanoTB.sv pela linha `initial memory = memoryA;`

Como o processador executa um programa?

Máquina de estados para sequenciamento das instruções e sinais de controle



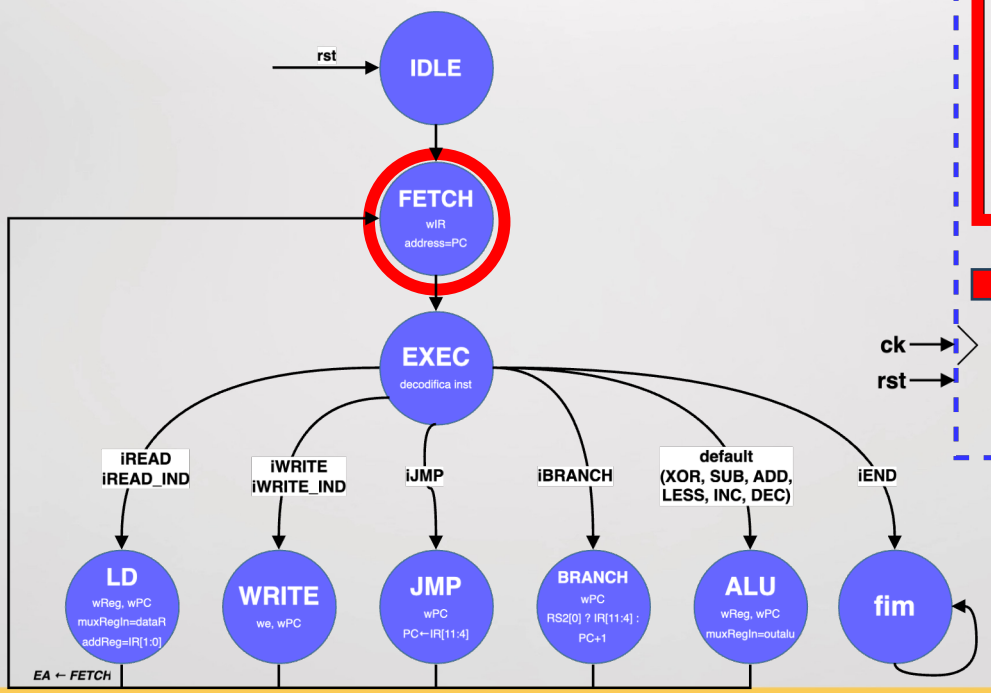
- Todas as instruções executam o estado de ***fetch*** (busca) para ler da memória a instrução apontada pelo **PC** e gravar este dado no registrador **IR**
- No estado ***exec*** a instrução é decodificada e define-se o próximo estado (a leitura da memória e a ULA são combinacionais)
- Depois cada instrução é concluída em um estado específico (por default as instruções lógicas e aritméticas são executadas no estado ***ALU***)
- O programa termina ao encontrar a instrução **fim**

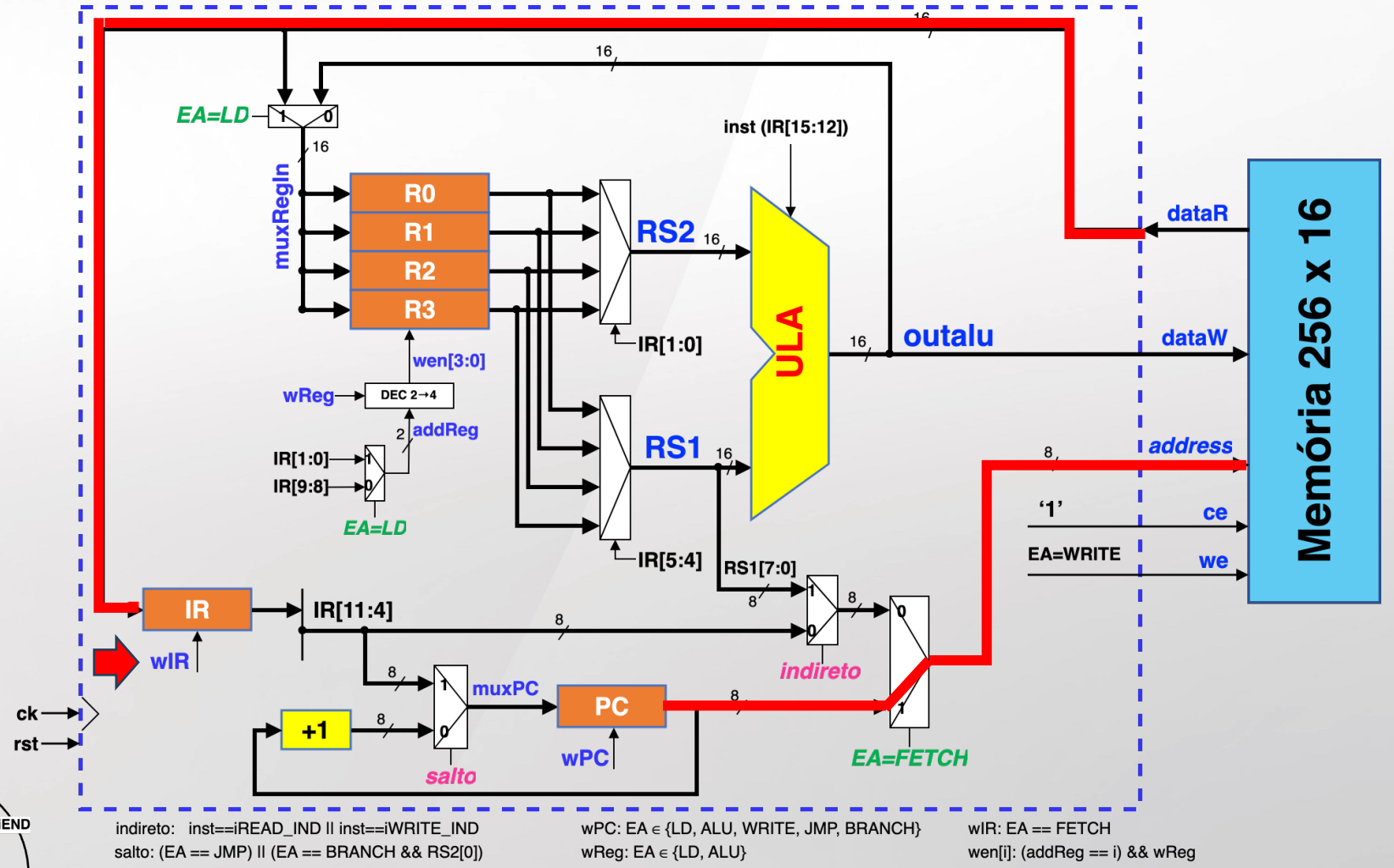
```
typedef enum logic [3:0] {
   IDLE, FETCH, EXEC, LD, WRITE,
   ALU, JMP, BRANCH, fim
} EAType;
EAType EA;
// apenas EA (código + simples)
```

Busca da instrução – igual para todas as instruções

Neste estado de ***fetch***:

- PC endereça a memória
- Dado é lido da memória
- Ativa o *wIR* para escrita em IR





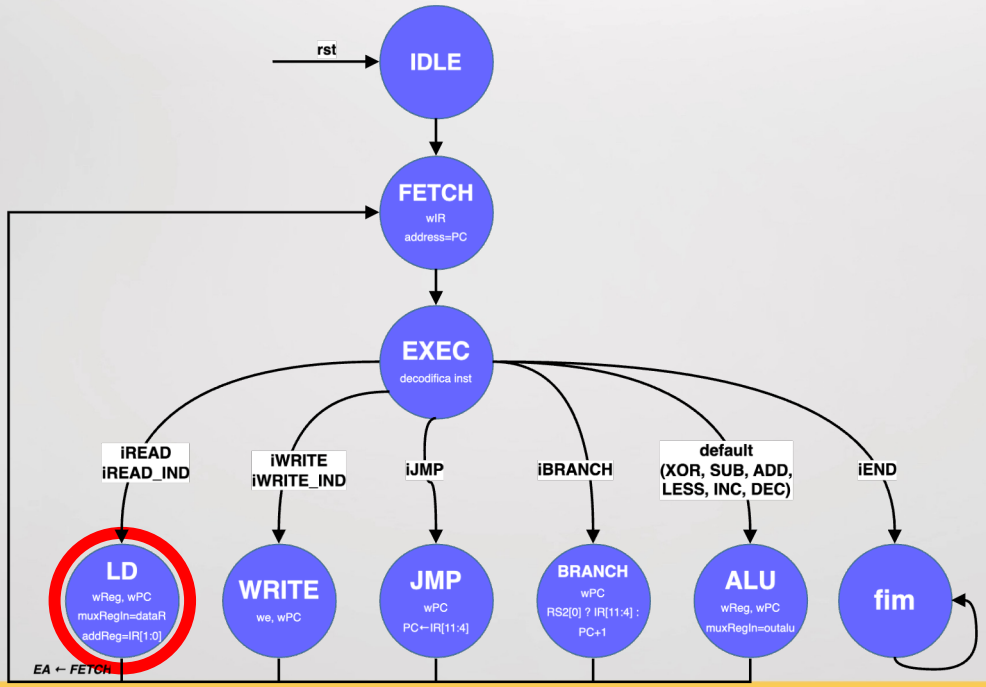
Busca da instrução - *fetch*

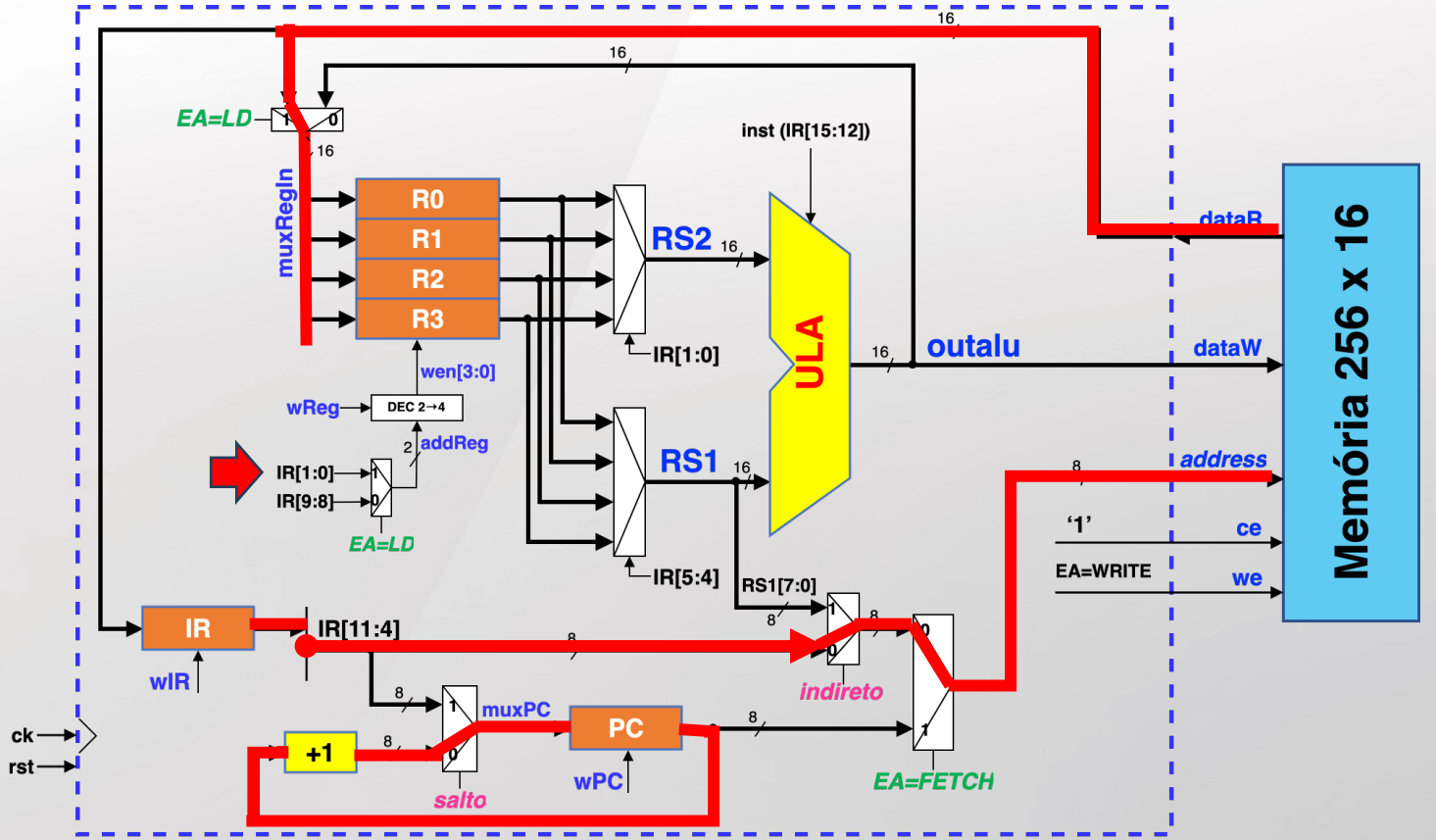
Execução do READ

No estado ***LD***:

- IR[11:4] endereça a memória
- Dado é lido da memória
- Escreve no banco de registradores ativando ***wReg*** (registrador alterado especificado em IR[1:0])
- Incrementa o PC (ativa ***wPC***)

Instrução	15:12	11:8	7:4	3:0
READ	0	endereço		Rs2



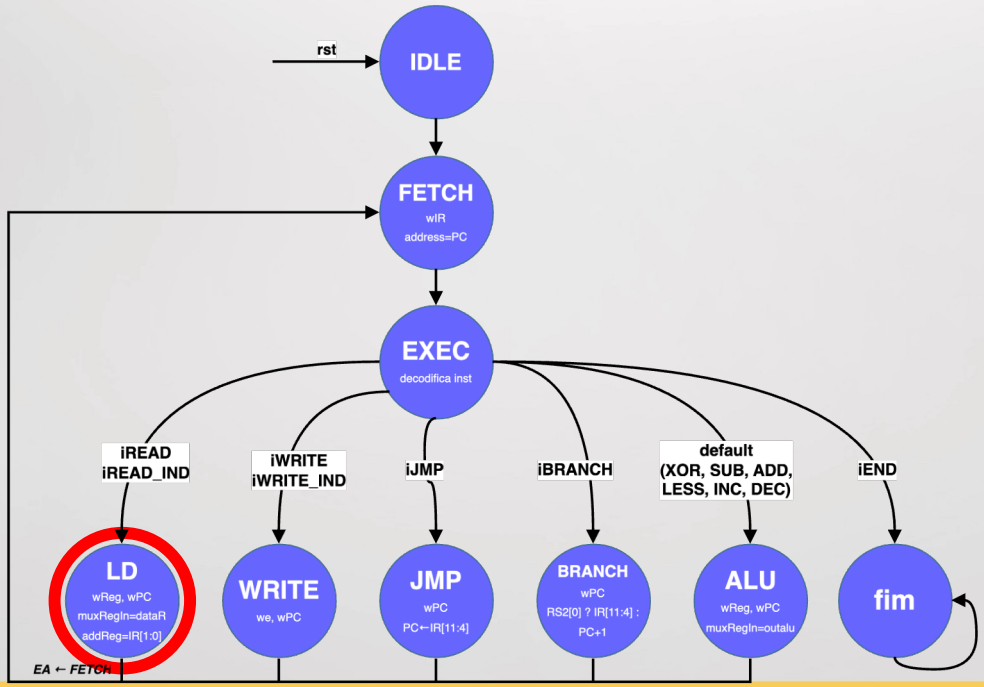


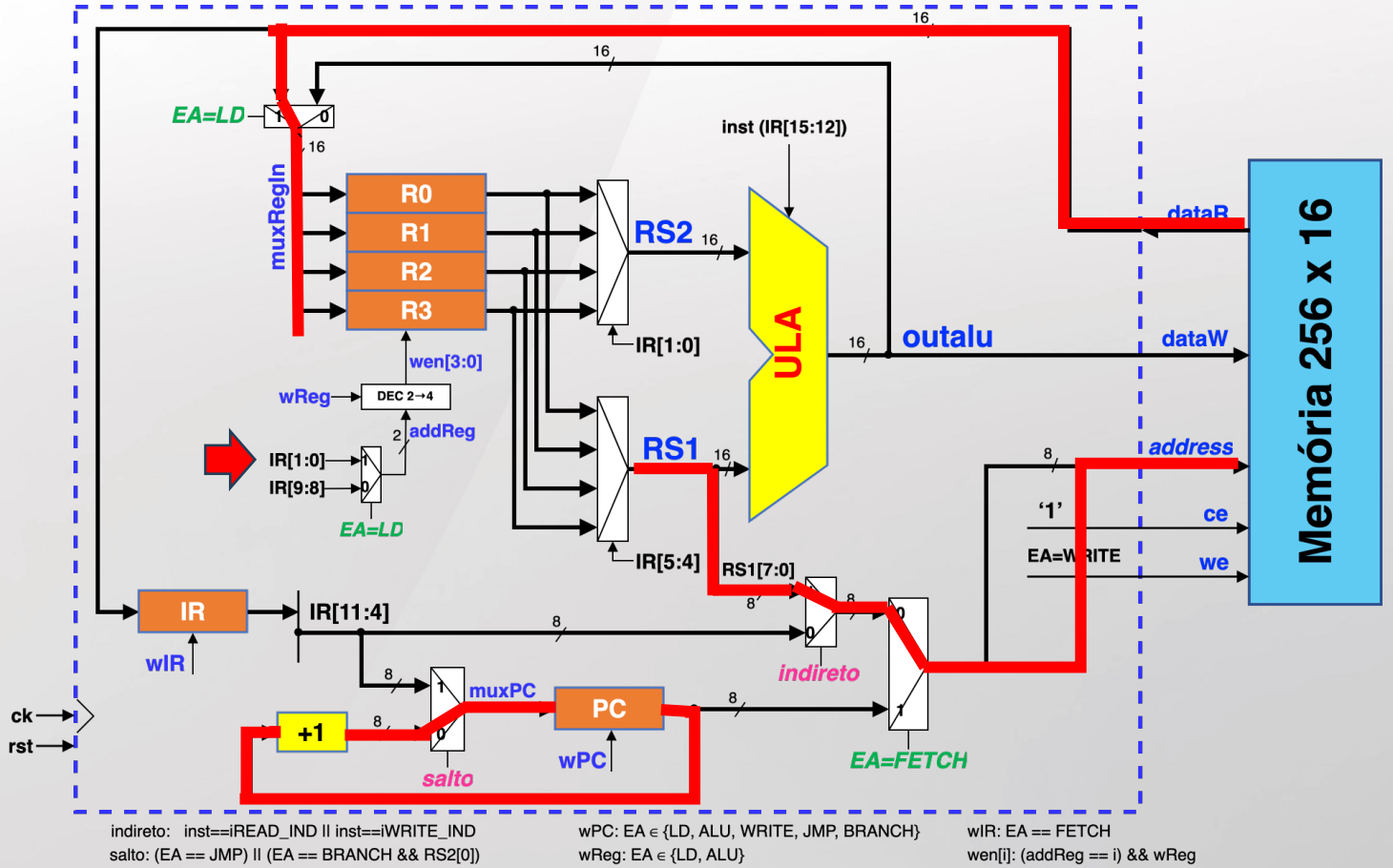
Execução do READ INDIRETO

No estado ***LD***:

- RS1[7:0] endereça a memória
- Dado é lido da memória
- Escreve no banco de registradores ativando ***wReg*** (registrador alterado especificado em IR[1:0])
- Incrementa o PC (ativa ***wPC***)

Instrução	15:12	11:8	7:4	3:0	Operação
READ_Indireto	10 $(A)_{16}$	0	Rs1	Rs2	Rs2 ← PMEM(**RS1**)





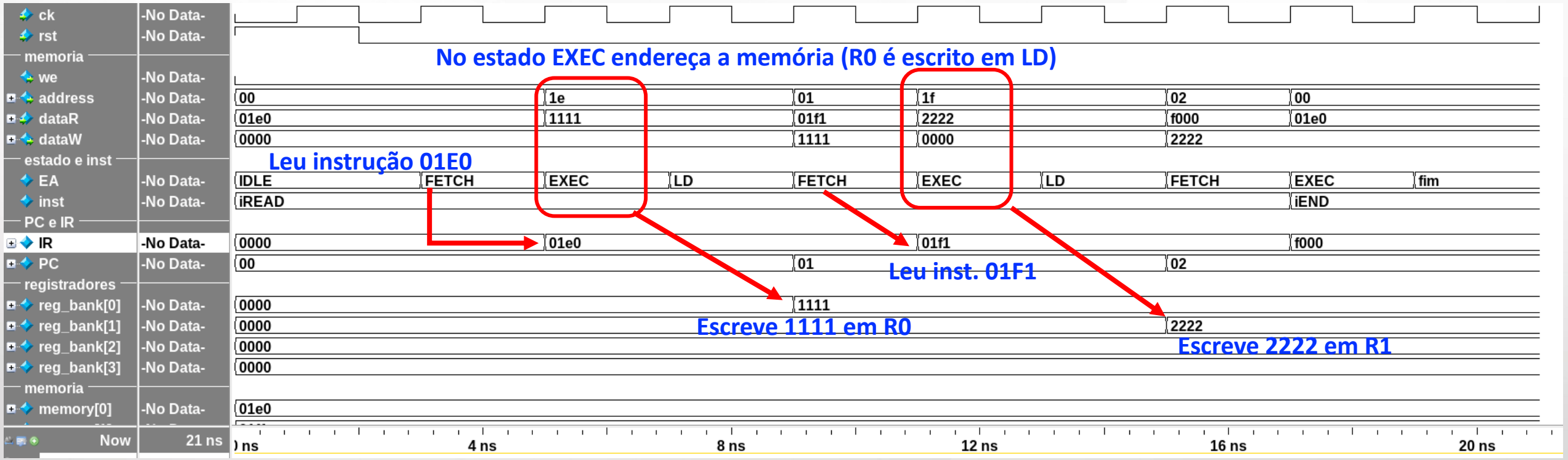
Simulação

- Baixar do Moodle o arquivo nano_cpu.zip

```
.
├── nanoCPU.sv
├── nanoTB.sv
├── sim.do
├── wave.do
└── Makefile
```

- Executar o sim.do fornecido – a janela *wave* é exibida, e no terminal são impressas mensagens de debug
  - O **Makefile** é para uso com outro simulador: Verilator
  - O programa simulado é escolhido no nanoTB.sv pela linha `initial memory = memoryA;`
  - O test bench também grava as formas de onda em `dump.fst` (abrir com `gtkwave dump.fst`)

Inicializando dois registradores




```
memory_array_t memoryB = '{
  0:  'h01E0, // R0 = PMEM[30]
  1:  'h01F1, // R1 = PMEM[31]
  2:  'hF000, // FIM
  30: 'h1111,
  31: 'h2222,
  default: 'h0000
};
```

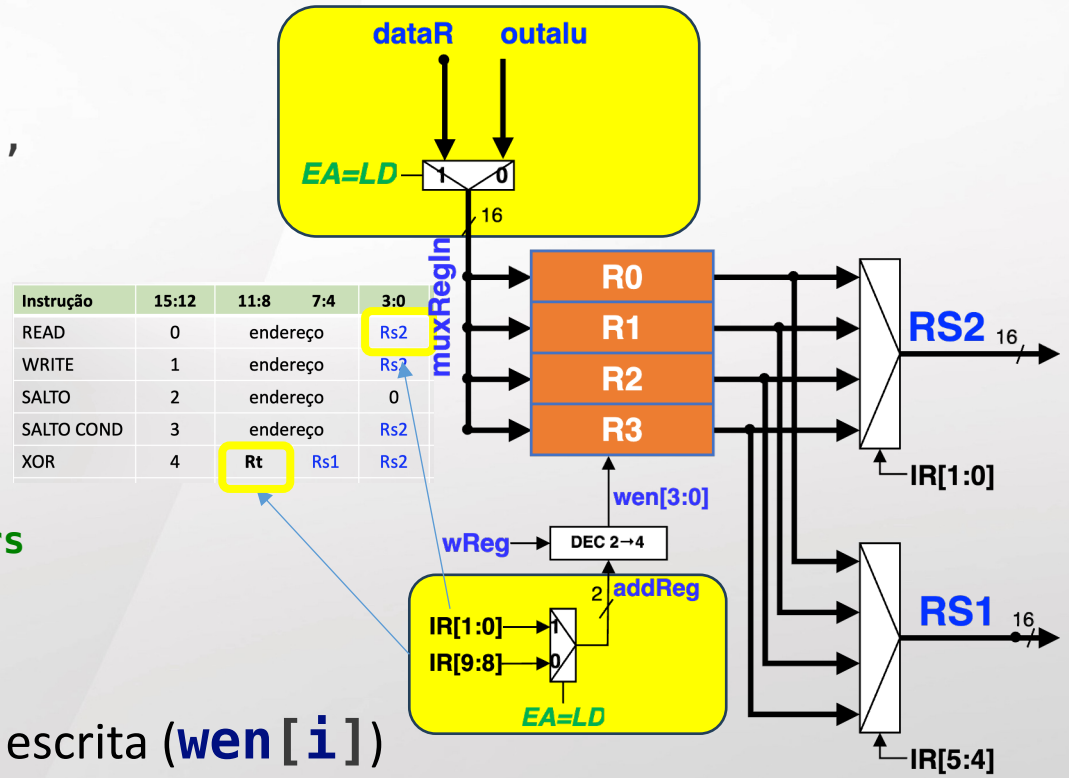
O memory_array armazena o binário (programa)
a estrutura é <endereço decimal> : <valor>

Atividade 1 completar o banco de registradores (a)

```
// register bank - 4 general purpose registers
generate
   for (genvar i = 0; i < 4; i++) begin
      Register #(16) reg_inst (.ck(ck), .rst(rst), .we(wen[i]),
                               .D(muxRegIn),.Q(reg_bank[i]) );
      assign wen[i] = addReg==i && wReg;
   end
endgenerate

assign addReg = IR[1:0];    // mux ausente
assign muxRegIn = dataR;    // mux ausente

assign RS1 = reg_bank[IR[5:4]]; // reg bank output multiplexers
assign RS2 = reg_bank[IR[1:0]];
```



Instrução	15:12	11:8	7:4	3:0
READ	0	endereço		Rs2
WRITE	1	endereço		Rs2
SALTO	2	endereço		0
SALTO COND	3	endereço		Rs2
XOR	4	Rt	Rs1	Rs2

- **generate**: instancia os 4 registradores e defina o sinal de escrita (**wen[i]**)

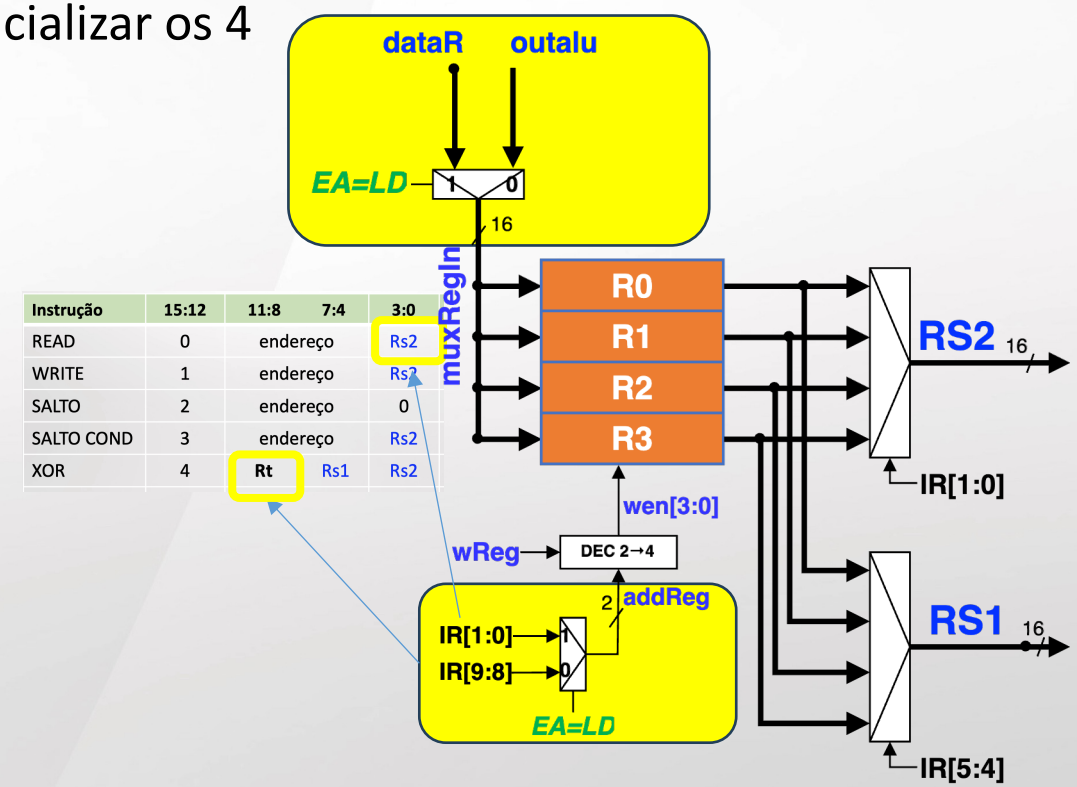
➔ **Completar os dois multiplexadores ausentes na descrição fornecida**

- **addReg** : deve receber IR[1:0] apenas no estado LD, por default deve receber IR[9:8]
- **muxRegIn** : deve receber o dado da memória (**dataR**) apenas no estado LD, por default deve receber **outalu**

Atividade 1: completar o banco de registradores (b)

Alterado o banco de registradores, executar o programa para inicializar os 4 registradores (alterando o *test bench*):

```
memory_array_t memoryC = '{
    0:  'h01E0,      // R0 = PMEM[30]
    1:  'h01F1,      // R1 = PMEM[31]
    2:  completar,   // R2 = PMEM[32]
    3:  completar,   // R3 = PMEM[33]
    4:  'hF000,      // FIM
    30: 'h1111,
    31: 'h2222,
    32: 'h3333,
    33: 'h4444,
    default: 'h0000
};
```

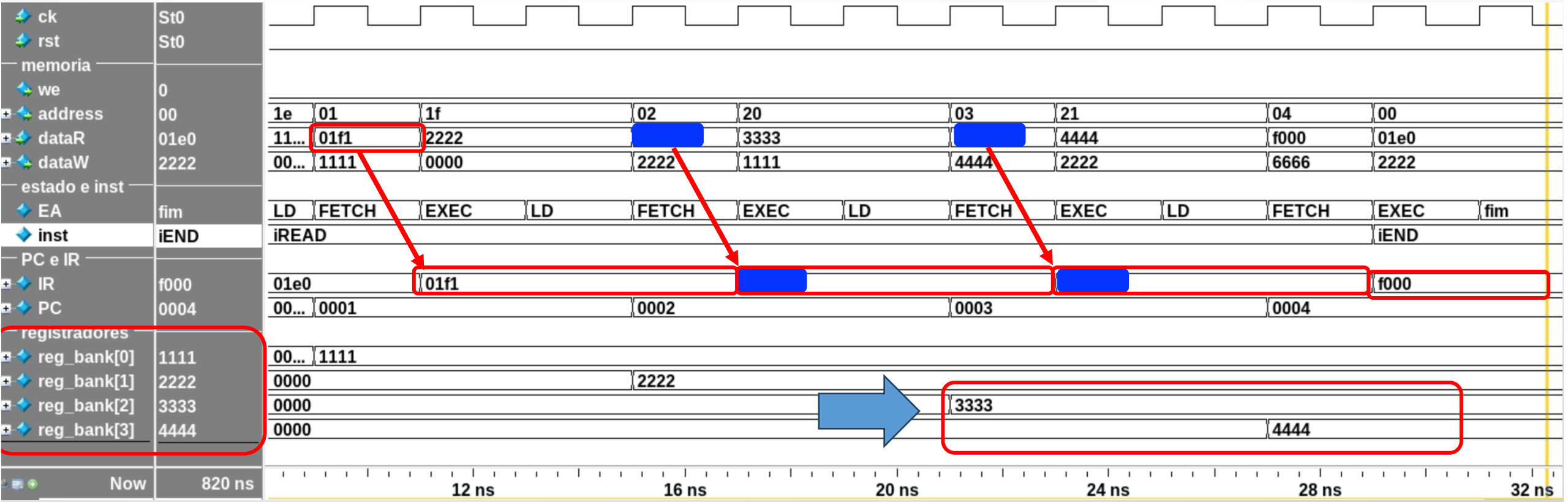


Instrução	15:12	11:8	7:4	3:0
READ	0	endereço		Rs2
WRITE	1	endereço		Rs2
SALTO	2	endereço		0
SALTO COND	3	endereço		Rs2
XOR	4	Rt	Rs1	Rs2

Instrução	15:12	11:8	7:4	3:0	Operação
READ	0	endereço		Rs2	Rs2 ← PMEM(end)
WRITE	1	endereço		Rs2	PMEM(end) ← Rs2
FIM	15 $(F)_{16}$	0	0	0	termina a execução

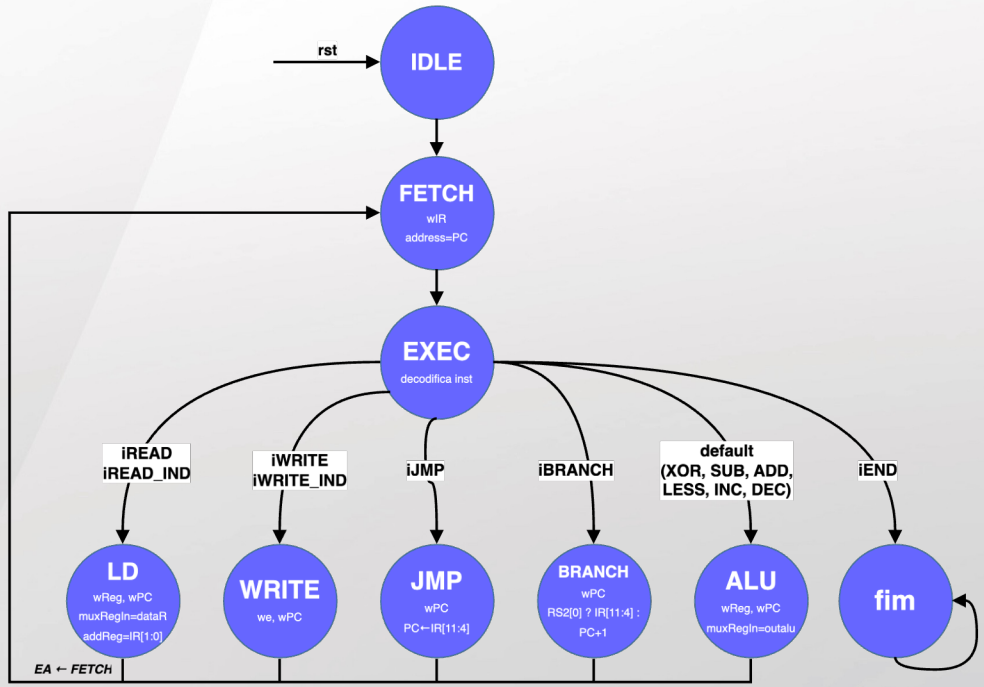
Atividade 1: completar o banco de registradores (c)

- Simular de forma a inicializar corretamente o **reg[2] e reg[3]** – zoom entre 0 e 35 ns



Atividade 2 completar a máquina de estados

```
always_ff @(posedge ck or posedge rst) begin
    if (rst)
        EA <= IDLE;
    else begin
        case (EA)
            IDLE:  EA <= FETCH;
            FETCH: EA <= EXEC;
            EXEC: case (inst)
                iEND:     EA <= fim;
                iREAD, iREAD_IND:     EA <= LD;
                iWRITE, iWRITE_IND:   EA <= completar;
                iJMP:     EA <= completar;
                iBRANCH:  EA <= completar;
                default: EA <= ALU;
              endcase
            fim: EA <= fim;
            default: EA <= FETCH;
        endcase
    end
end
```



Atividade 3 completar ULA

- **ULA** : a ULA fornecida só executa por default a soma. Faltam ao menos 3 instruções

```
// arithmetic and logic unit
always_comb begin
     case (inst)
        iWRITE, iWRITE_IND: outalu = RS2;
        // completar
        default: outalu = RS1 + RS2;
    endcase
end
```

Instrução	15:12	11:8	7:4	3:0	Operação
READ	0	endereço		Rs2	Rs2 ← PMEM(end)
WRITE	1	endereço		Rs2	PMEM(end) ← Rs2
SALTO	2	endereço		0	PC ← end
SALTO COND	3	endereço		Rs2	PC ← end **se** Rs2=1
XOR	4	**Rt**	Rs1	Rs2	Rt ← Rs1 xor Rs2
SUB	5	**Rt**	Rs1	Rs2	Rt ← Rs1 - Rs2
ADD	6	**Rt**	Rs1	Rs2	Rt ← Rs1 + Rs2
MENOR	7	**Rt**	Rs1	Rs2	Rt ← 1 **se** Rs1<Rs2 senão 0
...	...				*a ser incluído pelos alunos*
READ_Indireto	$10\ (A)_{16}$	0	Rs1	Rs2	Rs2 ← PMEM(**RS1**)
WRITE_Indireto	$11\ (B)_{16}$	0	Rs1	Rs2	PMEM(**RS1**) ← Rs2
FIM	$15\ (F)_{16}$	0	0	0	termina a execução

Dica: para **iLess (menor)**: outalu = (RS1 < RS2) ? 'h0001 : 'h0000;

Atividade 4 simulação para validar as alterações

- **Programa** de teste

Já temos:

R0 ← 1111

R1 ← 2222

R2 ← 3333

R3 ←4444

Vamos programar (acrescentar o código binário no testbench)

R0 ← R0 + R3 - espera-se 5555 em R0

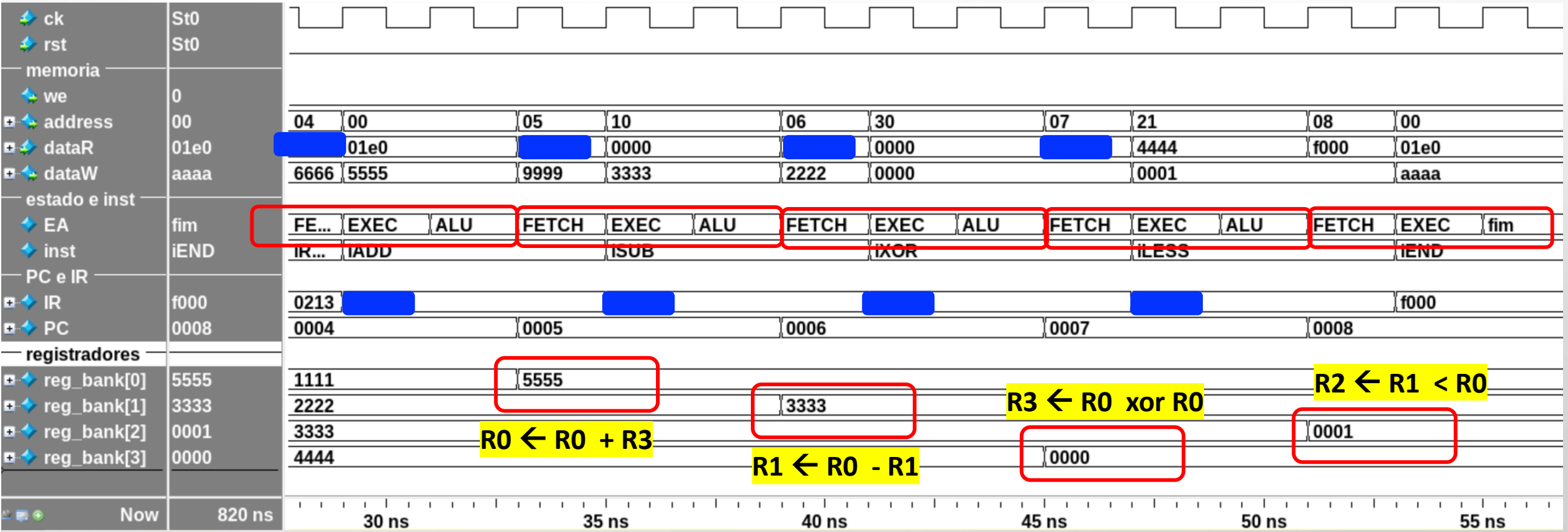
R1 ← R0 – R1 - espera 3333 em R1

R3 ← R0 xor R0 - espera-se que o R3 seja **zerado** (método comum para zerar registradores)

R2 ← R1 < R0 - 3333 < 5555, logo R2 deve ser 1

Atividade 4 simulação para validar as alterações

- Após fazer as alterações no código, e escrever as 4 instruções no *testbench* devemos obter:



Atividade 5 teste de escrita na memória

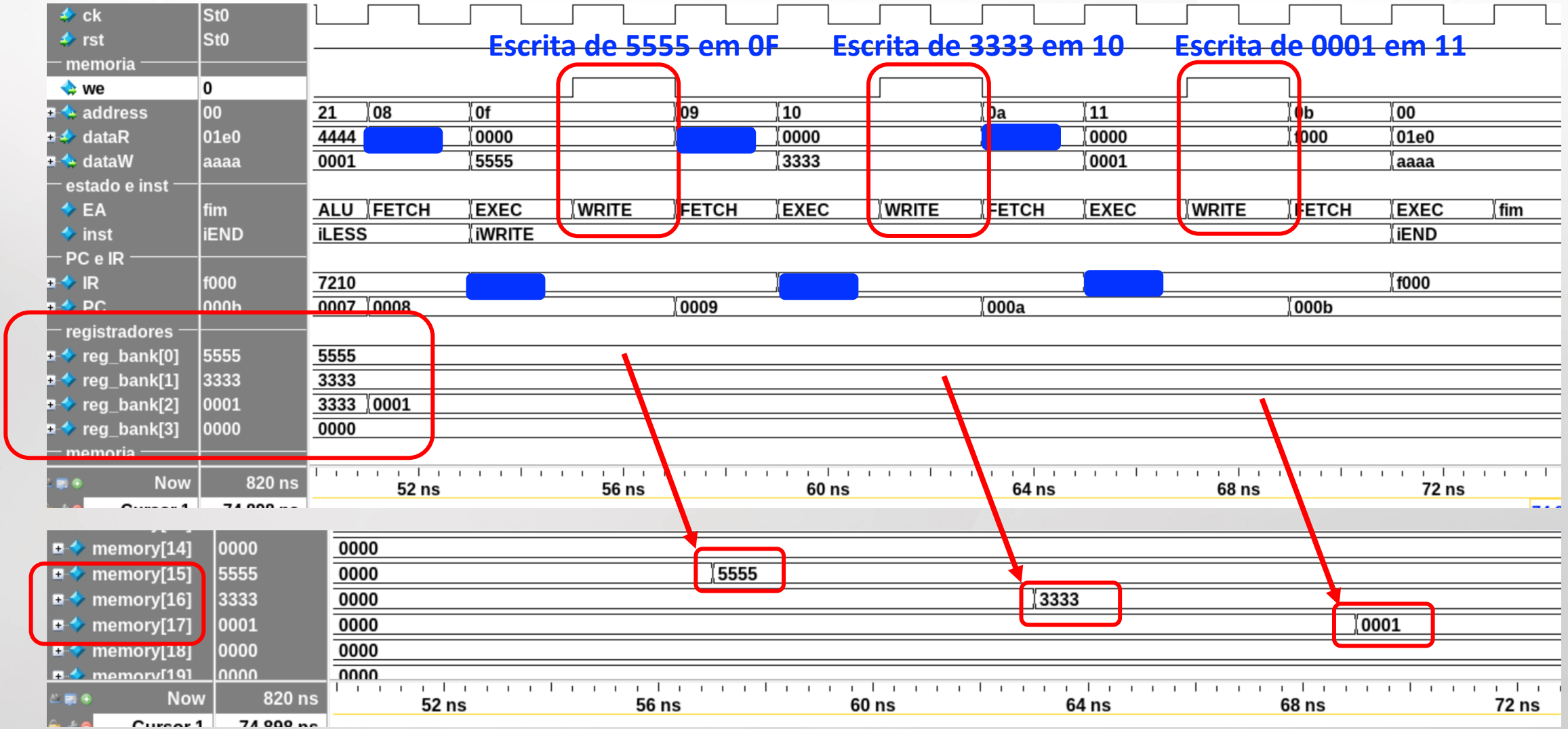
- O sinal **we** está assim definido no código fornecido:

```
assign we = 0; // atividade 5
```

- Este sinal deve ser ativo (1) quando o EA for WRITE, senão 0.

Atividade 5 teste de escrita na memória

Escrever o conteúdo dos registradores R0 (5555), R1 (3333), R2 (0001) na posições 15 $(0F)_{16}$, 16 $(10)_{16}$ e 17 $(11)_{16}$



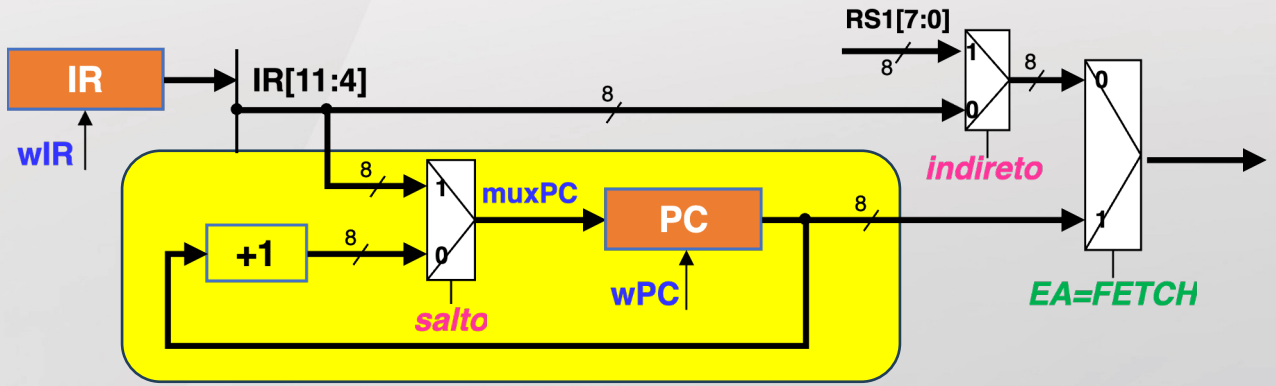
Atividade 6 — Saltos – condicionais e incondicionais

No código fornecido o PC está sendo sempre incrementado:

```
assign muxPC = PC + 1;
```

Instrução	15:12	11:8	7:4	3:0
READ	0	endereço		Rs2
WRITE	1	endereço		Rs2
SALTO	2	endereço		0
SALTO COND	3	endereço		Rs2
XOR	4	Rt	Rs1	Rs2

Deve-se implementar o muxpC de tal forma que o PC recebe os 8 bits do IR: `IR[11:4]`



Condições para a atribuição:

(c1) EA == JMP

(c2) EA == BRANCH && RS2[0] // bit do less ativo

```
assign muxPC = (c1 || c2 ) ? IR[11:4] : PC + 1;
```

Atividade 6: Saltos – condicionais e incondicionais

Lembrando: o R2 tem $(0001)_{16}$

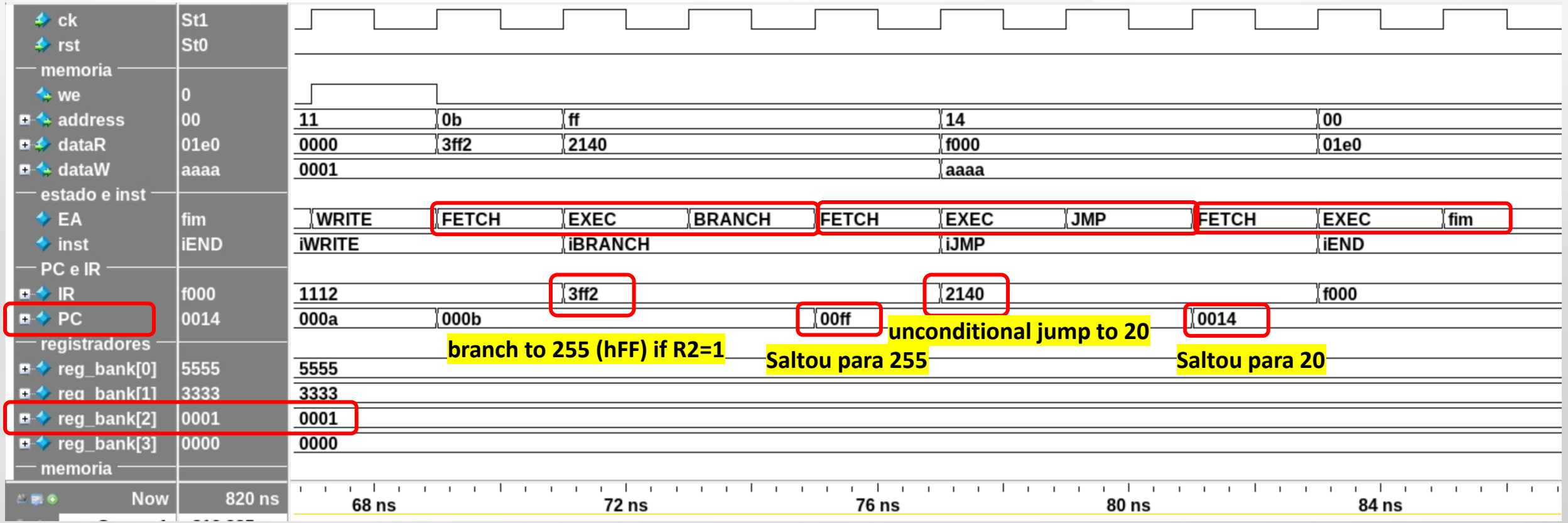
Após o armazenamento de R2:

- salto condicional para endereço 255 se R2=1 **(11=> 'h3FF2)**
- no endereço 255 salta para endereço 20 $(14)_{16}$ **(255=> 'h2140)**
- no endereço 20 termina o programa **(20=> 'hF000)**

Instrução	15:12	11:8	7:4	3:0
READ	0	endereço		Rs2
WRITE	1	endereço		Rs2
SALTO	2	endereço		0
SALTO COND	3	endereço		Rs2
XOR	4	**Rt**	Rs1	Rs2

```
    10: 'h1112, // store R2 in 11
    11: 'h3FF2, // branch to 255 (xFF) if R2=1
    20: 'hF000, // end of program
    30: 'h1111,
    31: 'h2222,
    32: 'h3333,
    33: 'h4444,
   255: 'h2140, // unconditional jump to 20 (x14)
default: 'h0000
};
```

Atividade 6: Saltos – condicionais e incondicionais

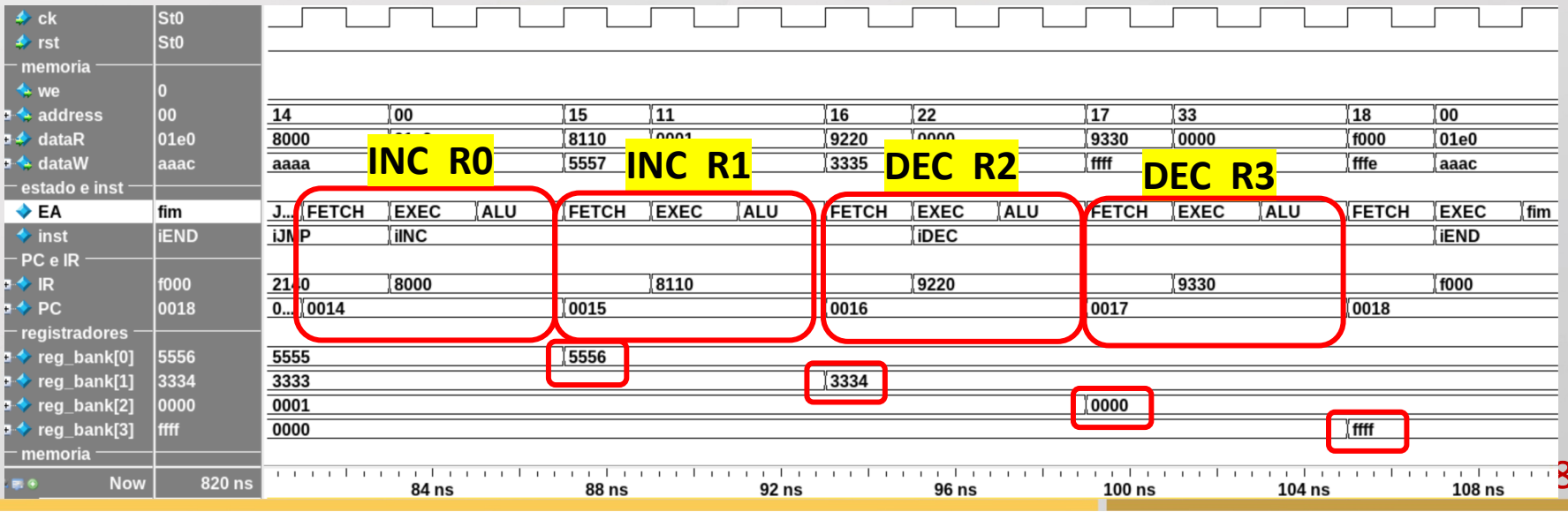


Atividade 7 Incluir instruções INC e DEC

- Incluir duas instruções – **iINC** e **iDEC**
- Usar os opcodes 8 e 9 para as instruções
  - INC: Rt ← RS1 +1, exemplo **'h8110** R1 ← R1 +1
  - DEC: Rt ← RS1 - 1, exemplo **'h9330** R3 ← R3 - 1
- Ações:
  1. **insere na lista de instruções – `instType` – `iINC, iDEC`**
  2. **acrescentar as operações de incremento e decremento na ULA**
  3. **A decodificação de `iINC` e `iDEC` é automática (cast para instType).**
- Programa teste:

```
11: 'h3FF2, // branch to 255 (xFF) if R2=1
20: 'h8000, // INC R0
21: 'h8110, // INC R1
22: 'h9220, // DEC R2
23: 'h9330, // DEC R3
24: 'hF000,
.....
```

Instrução	15:12	11:8	7:4	3:0	Operação
READ	0	endereço		Rs2	Rs2 ← PMEM(end)
WRITE	1	endereço		Rs2	PMEM(end) ← Rs2
SALTO	2	endereço		0	PC ← end
SALTO COND	3	endereço		Rs2	PC ← end **se** Rs2=1
XOR	4	**Rt**	Rs1	Rs2	Rt ← Rs1 xor Rs2
SUB	5	**Rt**	Rs1	Rs2	Rt ← Rs1 - Rs2
ADD	6	**Rt**	Rs1	Rs2	Rt ← Rs1 + Rs2
MENOR	7	**Rt**	Rs1	Rs2	Rt ← 1 **se** Rs1<Rs2 senão 0
...	...				*a ser incluído pelos alunos*
READ_Indireto	$10\ (A)_{16}$	0	Rs1	Rs2	Rs2 ← PMEM(**RS1**)
WRITE_Indireto	$11\ (B)_{16}$	0	Rs1	Rs2	PMEM(**RS1**) ← Rs2
FIM	$15\ (F)_{16}$	0	0	0	termina a execução



Atividade 8 executar o programa exemplo (a)

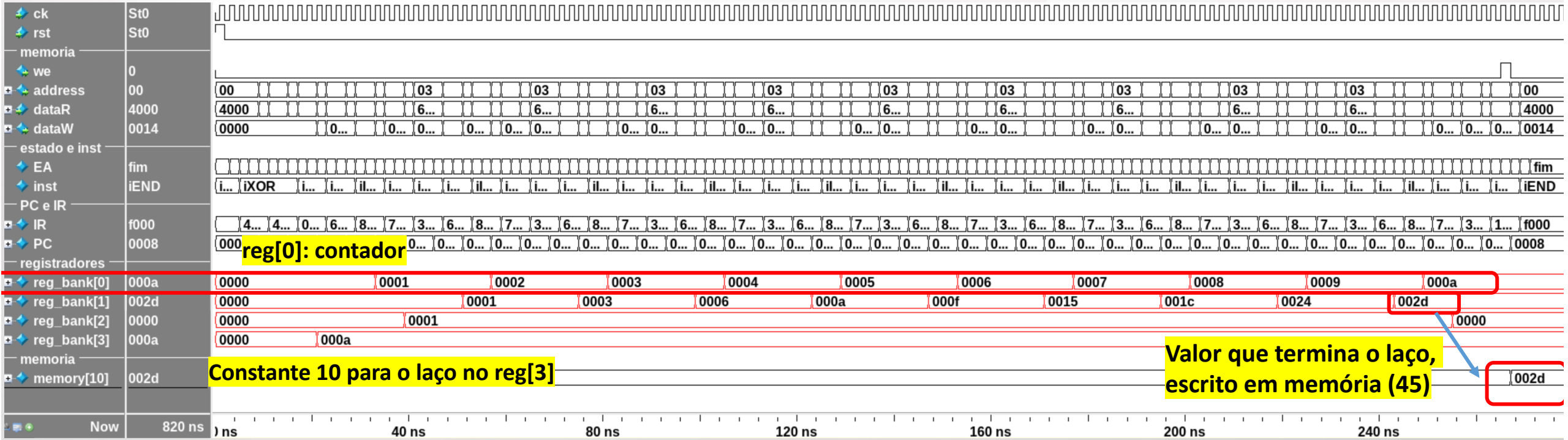
```
i = 0
a =0
do
      a = a +  i
      i  = i + 1
while (i<10)
“write” a
fim
```

```
memory_array_t memoryH = '{
      0: 'h4000,
      1: 'h4111,
      2: 'h0093,
      3: 'h6110,
      4: 'h8000,
      5: 'h7203,
      6: 'h3032,
      7: 'h10A1,
      8: 'hF000,
      9: 'h000A,
      default: 'h0000
};
```

Endereço	Instrução - *assembly*	comentário	**Binário (hexa)**
0	**xor** R0, R0, R0	R0 ←0 (i)	4 0 0 0
1	**xor** R1, R1, R1	R1 ←0 (a)	4 1 1 1
2	**read** R3, 9	Le da posição 09: R3 ← 10	0 09 3
3	**add** R1, R1, R0	a = a + i	6 1 1 0
4	**inc** R0, R0	R0 ← R0 + 1	8 0 0 0
5	**menor** R2, R0, R3	R2←1 se i<10 else 0	7 2 0 3
6	**salto cond** 3, R2	se R2=1 salta para 3	3 0 3 2
7	**write** R1, 10	Escreve a na posição 10	1 0A 1
8	**fim**	Termina a execução	F000
9	10		000A
10 (A)		Receberá o valor de a	

Atividade 8: executar o programa exemplo (b)

- Observar o **reg[0]** – variável i, conta de 0 a 9, no valor 10 termina o laço
- O **reg[1]** é o "**a = a + i**" – 0, 1, 3, 6, 10, 15, 21(x15), 28 (x1C), 36 (x24), 45 (x2D)
- No final grava na posição **10** da memória o valor 0x2D



Exemplos de programas

Sequência de Fibonacci

```
R1 = 0      // R1 = R1 xor R1  (fib1)
R2 = 1      // R2 = inc R0  (fib2)
R3 = N      // N = PMEM[20]

do {  R0 =0
      escreve fib1 na memória
      fib2 = fib1 + fib2
      fib1 = fib2 – fib1
      N = N -1
} while (N>0)
fim
```

Dicas:

- Dois registradores devem ser dedicados a fib1 (**R1**) e fib2 (**R2**)

fib2 (R2)	fib1 (R1)
1	**0**
1	**1**
2	**1**
3	**2**
5	**3**
8	**5**
13	**8**
21	**13**

```
memory_array_t fibonacci = '{
    …
    20: 'h000E, // 14 first elements of the series
    21: 'h0000, // receives the values of the series
    default: 'h0000
};
```

Sequência de Fibonacci

```
R1 = 0      // R1 = R1 xor R1  (fib1)
R2 = 1      // R2 = inc R0  (fib2)
R3 = N      // N = PMEM[20]

do {  R0 =0
      escreve fib1 na memória
      fib2 = fib1 + fib2
      fib1 = fib2 – fib1
      N = N -1
} while (N>0)
fim
```

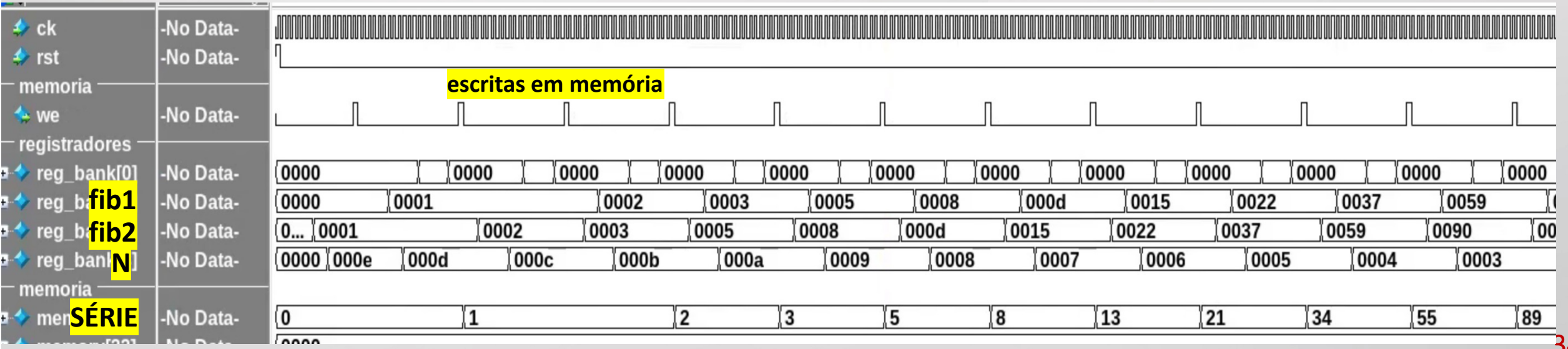
```
memory_array_t fibonacci = '{
        0: 'h4111, // (fib1) R1 = 0
        1: 'h8200, // (fib2) inc R2 (R2 = R0 + 1)
        2: 'h0143, // R3 = N (N = PMEM[20] = 14)
        // --- inicio do loop (endereco 3) ---
        3: 'h4000, // R0 = 0
        4: 'h1151, // write fib1 to memory
        5: 'h6212, // R2 = R1 + R2 (fib2 = fib1 + fib2)
        6: 'h5121, // R1 = R2 - R1 (fib1 = fib2 - fib1 => fib2 antigo)
        7: 'h9333, // dec N
        8: 'h7003, // R0 = 0 < N (verdadeiro para o loop)
        9: 'h3030, // conditional jump to 3 if R0[0] = 1
        // --- fim do loop ---
        10: 'hF000, // end
        20: 'h000E, // 14 first elements of the series
        21: 'h0000, // receives the values of the series
        default: 'h0000
};
```

Sequência de Fibonacci

```
R1 = 0      // R1 = R1 xor R1  (fib1)
R2 = 1      // R2 = inc R0  (fib2)
R3 = N      // N = PMEM[20]

do {  R0 =0
      escreve fib1 na memória
      fib2 = fib1 + fib2
      fib1 = fib2 – fib1
      N = N -1
} while (N>0)
fim
```

- **R0**: usado na comparação
- **R1**: fib1
- **R2**: fib2
- **R3**: N



Sequência de Fibonacci com escrita indireta

```
fib1 = 0  (R1)
fib2 = 1  (R2)
R0 = end_mem[21]  (*Vetor)
R3 = end_mem[20]  (N)
R3  = R0 + R3  (endereço final de final +1)
end_mem[22] = R3


do {
      escreve fib1 em *R0
      fib2 = fib1 + fib2
      fib1 = fib2 – fib1

       R0 = R0 + 1              // avança o ponteiro
       R3 = end_mem[22]         // le da memória o valor final

} while (R0< R3)
```

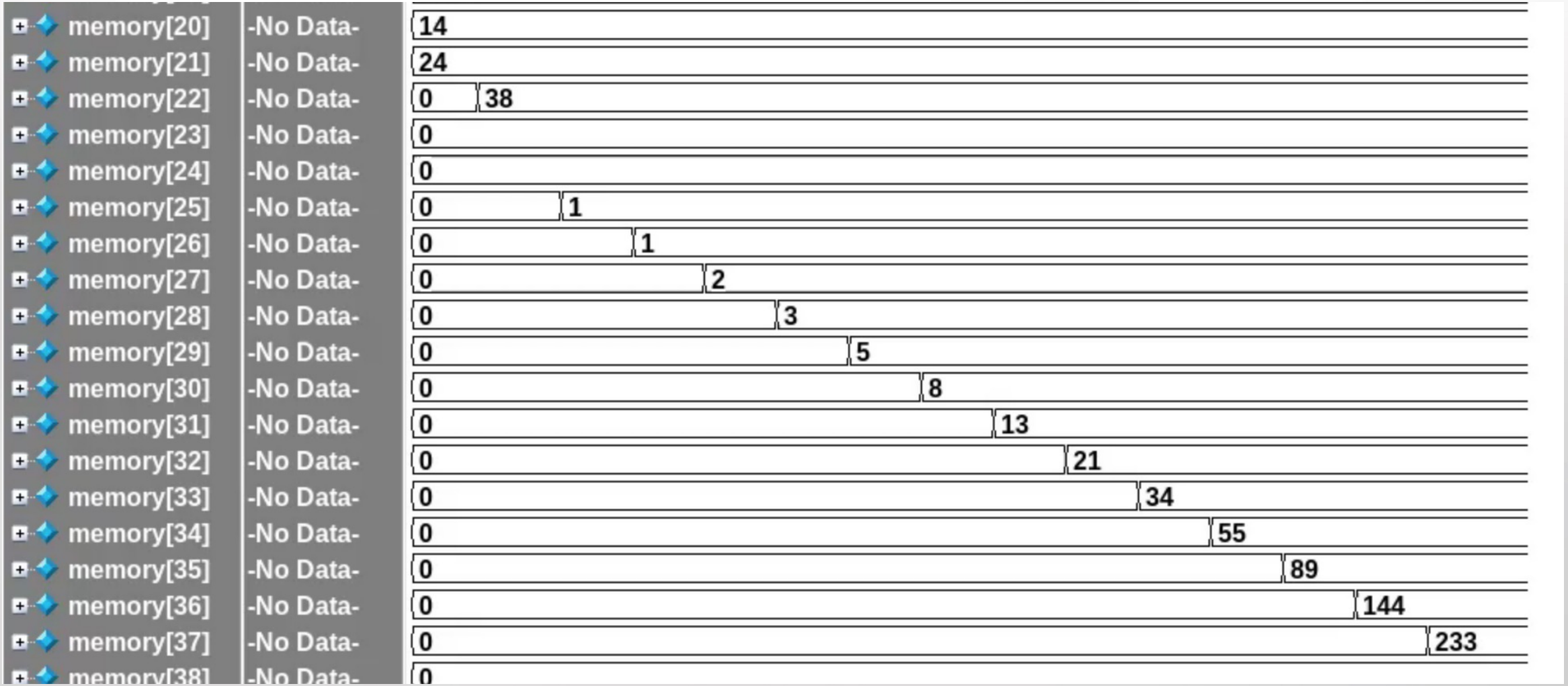
Problema:

- Apenas 4 registradores

Solução:

- Usar posições de memória como valores temporários

Sequência de Fibonacci com escrita indireta



Soma de constante a um vetor (somacteV)

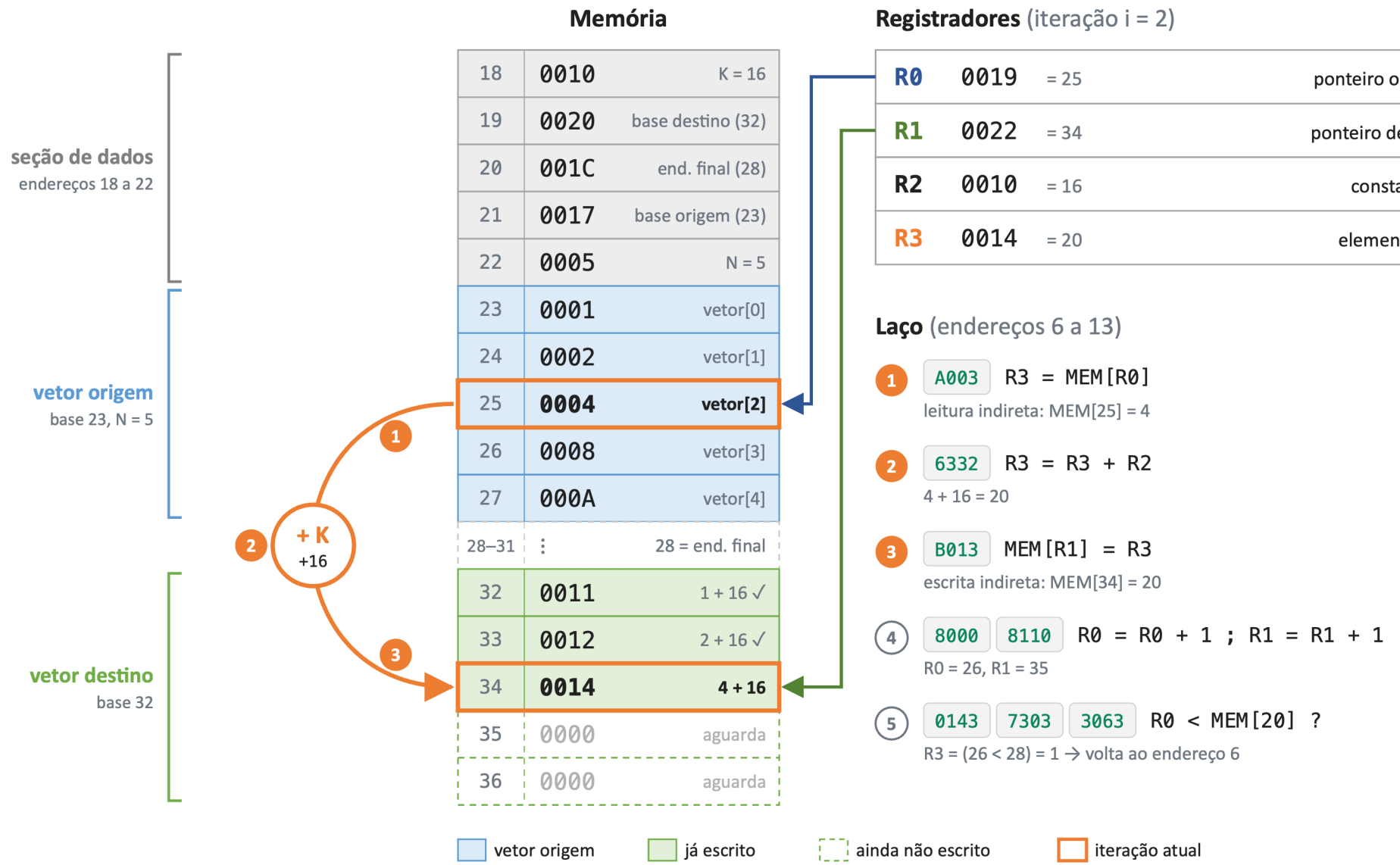
```
R0 = MEM[21]   // *origem
R2 = MEM[22]   // N
R2 = R0 + R2   // end. final
MEM[20] = R2
R1 = MEM[19]   // *destino
R2 = MEM[18]   // K

do {
   R3 = *R0
   R3 = R3 + K
   *R1 = R3
   R0 = R0 + 1
   R1 = R1 + 1
   R3 = MEM[20]  // end. final
} while (R0 < R3)
fim
```

```
memory_array_t somacteV = '{
  0: 'h0150, // R0 = MEM[21]   (ponteiro origem)
  1: 'h0162, // R2 = MEM[22]   (N elementos)
  2: 'h6202, // R2 = R0 + R2   (endereço final)
  3: 'h1142, // MEM[20] = R2   (guarda end. final)
  4: 'h0131, // R1 = MEM[19]   (ponteiro destino)
  5: 'h0122, // R2 = MEM[18]   (constante K)
// --- início do laço (endereço 6) ---
  6: 'hA003, // R3 = MEM[R0]   (leitura indireta)
  7: 'h6332, // R3 = R3 + R2   (soma K)
  8: 'hB013, // MEM[R1] = R3   (escrita indireta)
  9: 'h8000, // R0 = R0 + 1    (avança origem)
 10: 'h8110, // R1 = R1 + 1    (avança destino)
 11: 'h0143, // R3 = MEM[20]   (endereço final)
 12: 'h7303, // R3 = (R0 < R3) ? 1 : 0
 13: 'h3063, // if R3[0]=1 goto 6
// --- fim do laço ---
 14: 'hF000, // FIM
   ...   // seção de dados no próximo slide
};
```

```
R0 = MEM[21]   // *origem
R2 = MEM[22]   // N
R2 = R0 + R2   // end. final
MEM[20] = R2
R1 = MEM[19]   // *destino
R2 = MEM[18]   // K

do {
    R3 = *R0
    R3 = R3 + K
    *R1 = R3
    R0 = R0 + 1
    R1 = R1 + 1
    R3 = MEM[20]  // end. final
} while (R0 < R3)
fim
```



somacteV – dados e resultado esperado

R0: ponteiro ao vetor origem
R1: ponteiro ao vetor destino
R2: constante K (no laço)
R3: elemento lido / endereço final

Problema:
apenas 4 registradores

Solução:
o endereço final (base + N) fica em MEM[20] e é relido a cada iteração

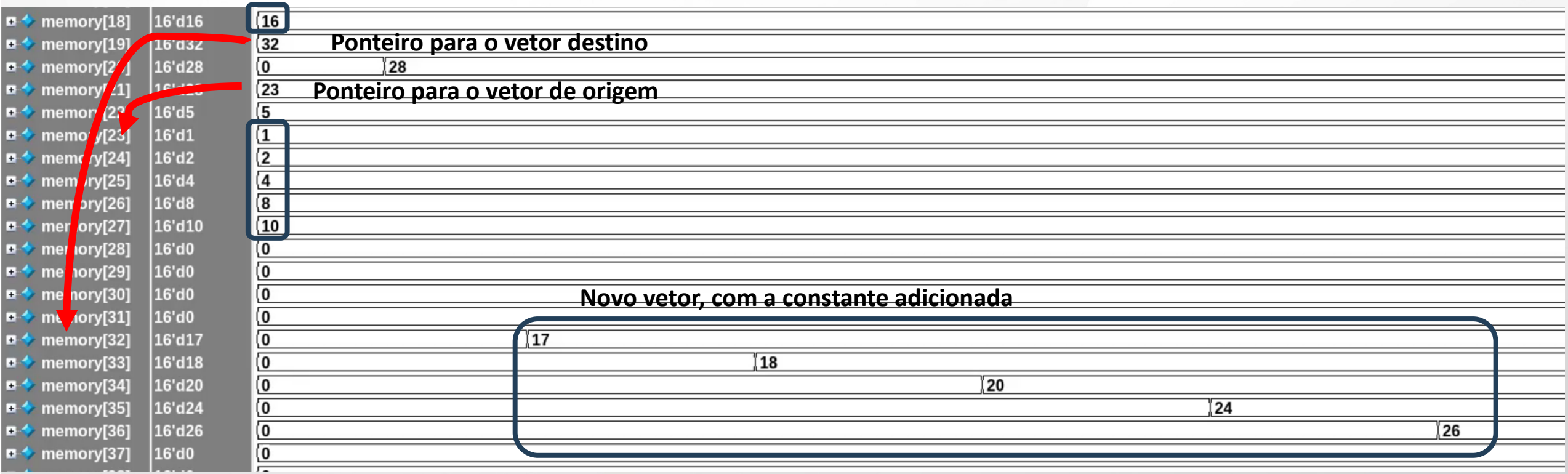
Endereçamento indireto:
iREAD_IND (A) e iWRITE_IND (B) usam RS1[7:0] como endereço

```
// seção de dados (antes da execução)
 18: 'h0010, // K = 16
 19: 'h0020, // base do vetor destino (32)
 20: 'h0000, // end. final (calculado)
 21: 'h0017, // base do vetor origem (23)
 22: 'h0005, // N = 5 elementos
 23: 'h0001, // vetor[0]
 24: 'h0002, // vetor[1]
 25: 'h0004, // vetor[2]
 26: 'h0008, // vetor[3]
 27: 'h000A, // vetor[4]

// resultado esperado (após FIM)
 20: 'h001C, // 23 + 5 = 28
 32: 'h0011, // 1 + 16 = 17
 33: 'h0012, // 2 + 16 = 18
 34: 'h0014, // 4 + 16 = 20
 35: 'h0018, // 8 + 16 = 24
 36: 'h001A, // 10 + 16 = 26
```

somacteV – dados e resultado esperado

Resultado da simulação



Divisão por subtrações sucessivas

```
R1 ← dividendo   (lê da posição de memória 20)
R2 ← divisor     (lê da posição de memória 21)

if R2 < 1 then                      // divisor igual a zero
   goto FIM
end if

R0 ← 0                              // zera o quociente
LOOP:  R1 ← R1 – R2                 // subtração

        if R1 < 0 then              //  negativo indica final de divisão
             goto SAVE
        else
             R0 ← R0 + 1            // incrementa quociente
             goto LOOP
        end if

SAVE:  R1 ← R1 + R2    // restaura dividendo → agora R1 é o resto
       grava o R0 (quociente) na memória (exemplo: endereço 22)
       grava o R1 (resto) na memória (exemplo: endereço 23)
FIM:
```

```
memory_array_t divisao = '{

0: 'h0141, // R1 <- PMEM(20) Le o dividendo
……..

20: 'd100,
21: 'd22,
22: 'h0000,
23: 'h0000,
default: 'h0000
};
```

Exemplo: divisão de 17 por 5

Estado inicial:

R1 = 17
R2 = 5
R0 = 0 (**quociente**)

Execução (**R1 ← R1 – R2**):

17 – 5 = 12 → R0 = 1
12 – 5 = 7 → R0 = 2
7 – 5 = 2 → R0 = 3
2 – 5 = -3 → valor negativo → salta para SAVE

SAVE: (restauração):

R1 = -3 + 5 = **2** (resto)
R0 = **3** (quociente)

PMEM(22) ← R0
PMEM(23) ← R1

```
R0 ← 0
LOOP:  R1 ← R1 – R2

       if R1 < 0 then
           goto SAVE
       else
           R0 ← R0 + 1
           goto LOOP
       end if

SAVE:  R1 ← R1 + R2
       grava o R0 (quociente) na memória
       grava o R1 (resto) na memória
```

ALTERAR A INSTRUÇÃO LESS (ULA):

```
iLESS: outalu = ($signed(RS1) < $signed(RS2)) ? 'h0001 : 'h0000;
```

```
memory_array_t divisao = '{

  0: 'h0141,  // R1 <- PMEM(20)          Le em R1 o dividendo
  1: 'h0152,  // R2 <- PMEM(21)          Le em R2 o divisor

  2: 'h4000,  // xor R0, R0, R0
  3: 'h8000,  // inc R0
  4: 'h7320,  // R3 <- R2 < 1
  5: 'h3103,  // salta para fim          se divisor <1 salta para fim

  6: 'h4000,  // xor R0, R0, R0         zera o quociente (R0)

  7: 'h5112,  // R1 <- R1 - R2      laço:   dividendo = dividendo - divisor (R1=R1-R2)

  8: 'h4333,  // xor R3, R3, R3
  9: 'h7313,  // R3 <- R1 < 0           teste se R1 < 0
 10: 'h30D3,  // salta para 13           se dividendo < 0 salta para save

 11: 'h8000,  // inc R0                 incrementa o quociente (R0)
 12: 'h2070,  // salta para 7            salta para a laço

 13: 'h6112,  // recupera o resto        dividendo = dividendo + divisor (será o resto)
 14: 'h1160,  // PMEM(22) <- R0          grava o quociente
 15: 'h1171,  // PMEM(23) <- R1          grava o dividendo (resto)

 16: 'hF000,  // fim                fim:

 20: 'd100,
 21: 'd22,
 22: 'h0000,
 23: 'h0000,
default: 'h0000  };
```

R1 ← dividendo (lê da posição de memória 20)
R2 ← divisor (lê da posição de memória 21)

if R2 < 1 then goto FIM

R0 ← 0

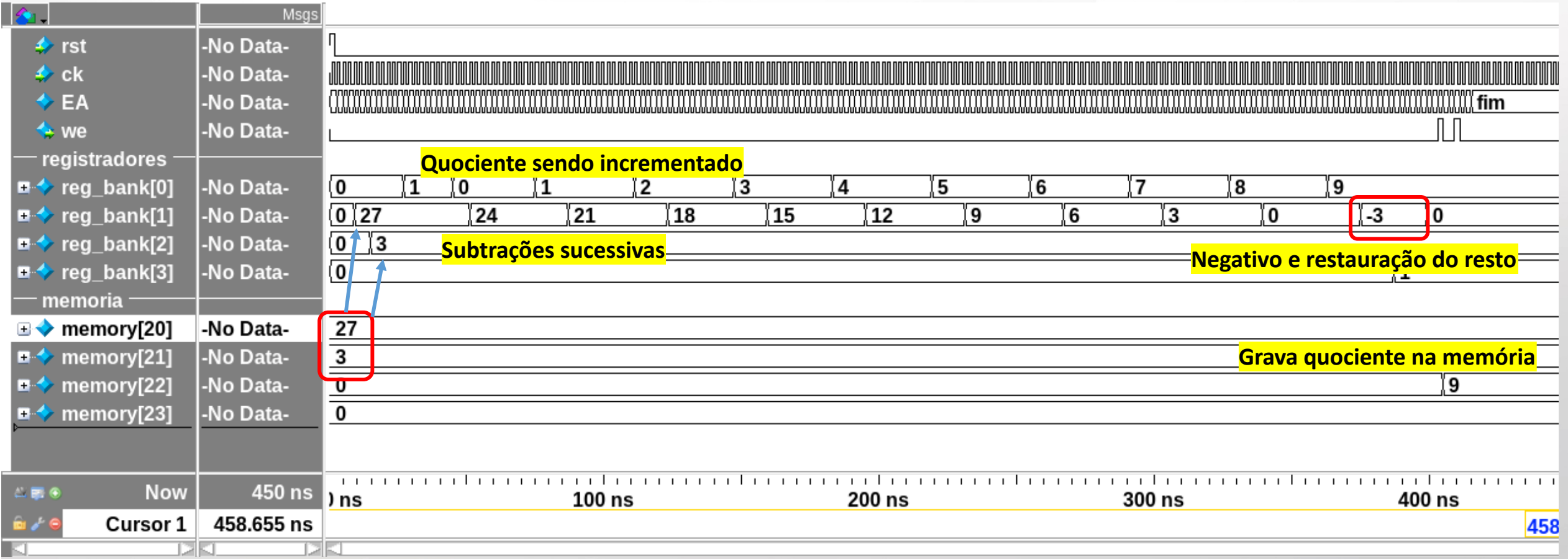
```
LAÇO :   R1 ← R1 – R2                    // subtração

         if R1 < 0 then                  //  negativo indica final de divisã
            goto SAVE
         else
            R0 ← R0 + 1                  // incrementa quociente
            goto LAÇO
         end if

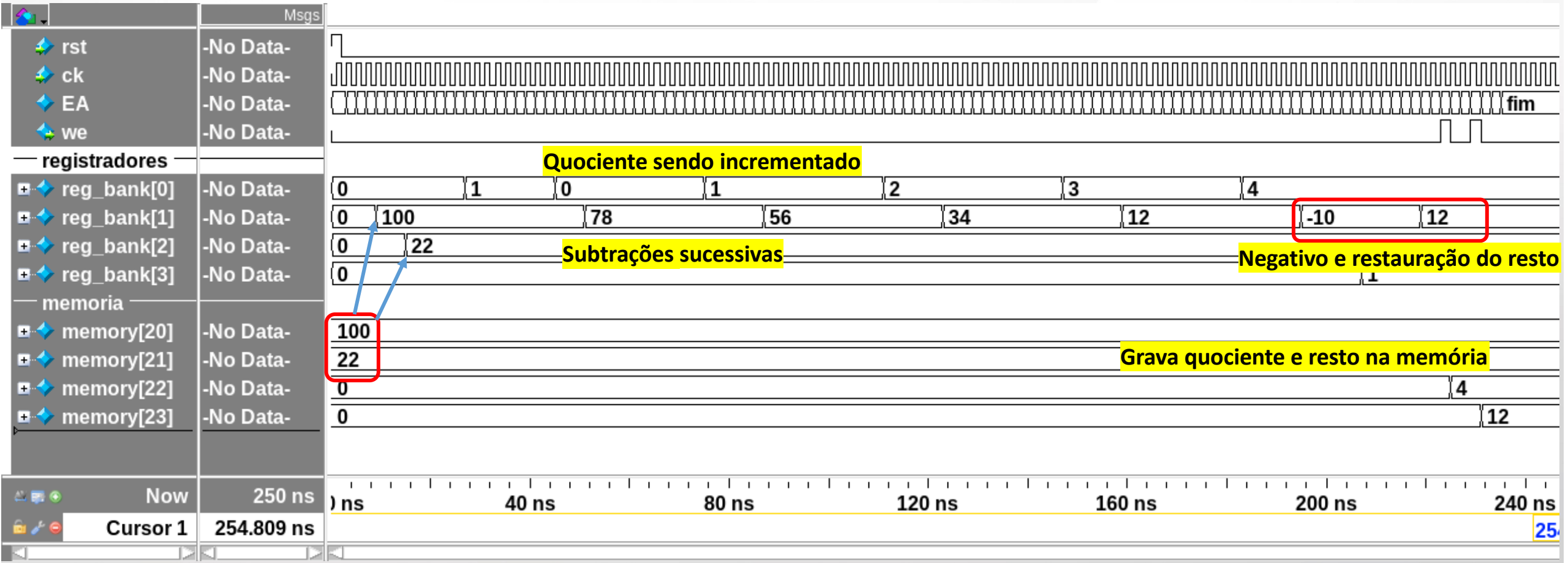
SAVE:    R1 ← R1 + R2    // restaura dividendo → agora R1 é o resto
         grava o R0 (quociente) na memória (exemplo: endereço 22)
         grava o R1 (resto) na memória (exemplo: endereço 23)

FIM:
```

Exemplo: divisão de 27 por 3

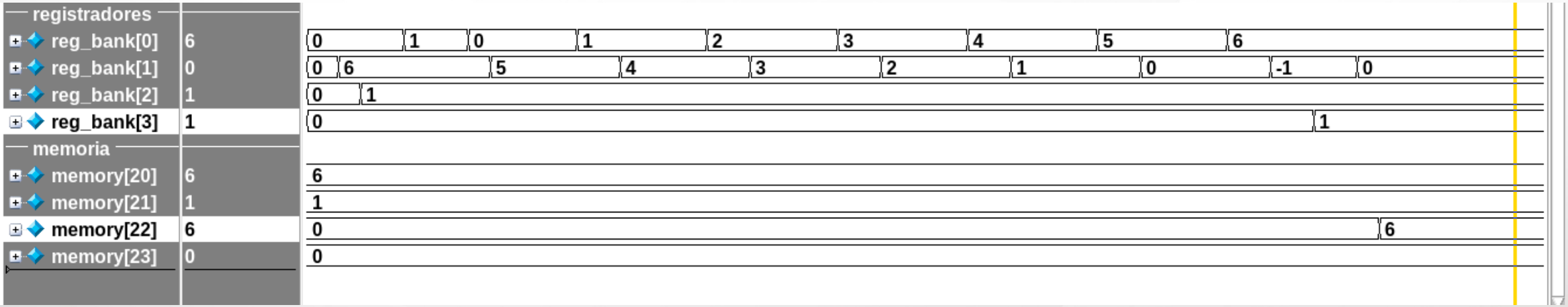


Exemplo: divisão de 100 por 22

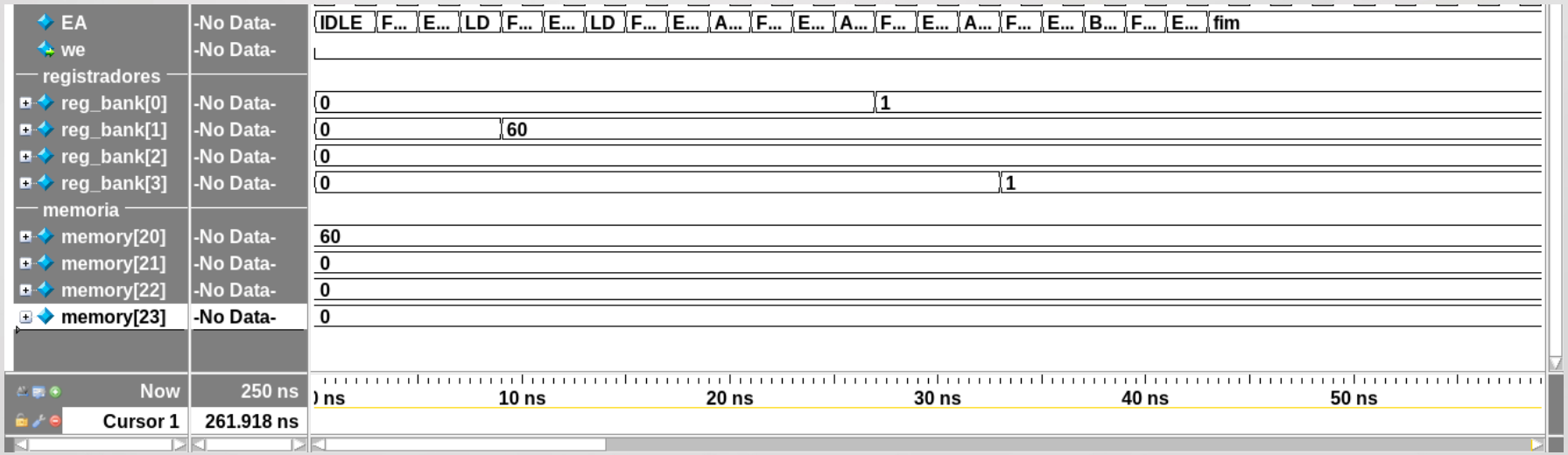


Exemplos: divisão por 1 e divisão por 0

Divisão de 6 por 1 – iterações até R1=-1



Divisão por 0 – programa termina após o teste R2 < 1



FIM DA UNIDADE 4